

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
МЕХАНИКО-МАТЕМАТИЧЕСКИЙ ФАКУЛЬТЕТ
Кафедра веб-технологий и компьютерного моделирования

ВЫЧИСЛИТЕЛЬНАЯ ПРАКТИКА

ОТЧЁТ

[ФИО СТУДЕНТА]
студента 2-го курса,
специальность «Математика и компьютерные науки»

Руководитель:
старший преподаватель Аленский Н. А.

Минск, 2026

СОДЕРЖАНИЕ

Обновите поле оглавления (F9)

1. ЦЕЛОЧИСЛЕННАЯ АРИФМЕТИКА И ОДНОМЕРНЫЙ МАССИВ

1.1. ПОСТАНОВКА ЗАДАНИЯ

Условие. В заданной последовательности натуральных чисел размера n найти наибольшее число, для которого сумма минимальной и максимальной цифр его десятичной записи является чётным числом. Массив хранить в динамически выделяемой памяти; работать с числами без преобразования в строку и без дополнительного массива цифр.

Ограничения и требования к программе:

- использовать только учебный материал лекций;
- не создавать дополнительный массив для хранения цифр числа;
- не задавать число как символьный массив, не использовать объекты класса `string`;
- все повторяющиеся действия оформить в виде функций;
- хранить последовательность в динамическом массиве `int*`;
- результат выводить в доступном и понятном виде.

1.2. АЛГОРИТМ И ПРОГРАММА

Программа реализована в классе `NumberArray`. Конструктор запрашивает размер, выделяет массив через `new int[size]` и вызывает `inputArray`. Метод `findAnswer` выполняет линейный поиск: для каждого элемента вычисляются минимальная и максимальная цифры и проверяется чётность их суммы. Среди подходящих чисел выбирается наибольшее. Деструктор освобождает память через `delete[]`.

1.2.1. Класс *NumberArray* и работа с памятью

Заголовочный файл описывает интерфейс класса: приватные поля (указатель на массив и его размер) и публичные методы конструктора, деструктора и запуска алгоритма.

```
class NumberArray
```

```

{
private:
    int* array;
    int size;

    void inputArray();
    void printArray() const;

    int getMinDigit(int number) const;
    int getMaxDigit(int number) const;

    bool isSuitable(int number) const;

    int findAnswer() const;

public:
    NumberArray();
    ~NumberArray();

    void run();
};

#endif

```

Конструктор и деструктор отвечают за жизненный цикл динамического массива:

```

NumberArray::NumberArray()
{
    std::cout << "Enter array size: ";
    std::cin >> size;

    array = new int[size];

    inputArray();
}

NumberArray::~NumberArray()
{
    delete[] array;
}

```

1.2.2. Функции *inputArray* и *printArray*

Ввод и вывод элементов выполняются через указатель `ptr`, который перемещается от начала массива до `array + size`. Это демонстрирует прямую работу с адресной арифметикой.

```

void NumberArray::inputArray()
{
    std::cout << "Enter elements:\n";

    int* ptr = array;

    while (ptr < array + size)
    {
        std::cin >> *ptr;
        ++ptr;
    }
}

void NumberArray::printArray() const
{
    const int* ptr = array;

    while (ptr < array + size)
    {
        std::cout << *ptr << " ";
        ++ptr;
    }

    std::cout << '\n';
}

```

1.2.3. Функции *getMinDigit* и *getMaxDigit*

Обе функции извлекают цифры целочисленным делением на 10 и сравнивают их с текущим экстремумом. Начальное значение `minDigit` равно 9, `maxDigit` — 0, что гарантирует обновление при первом же шаге для положительных чисел.

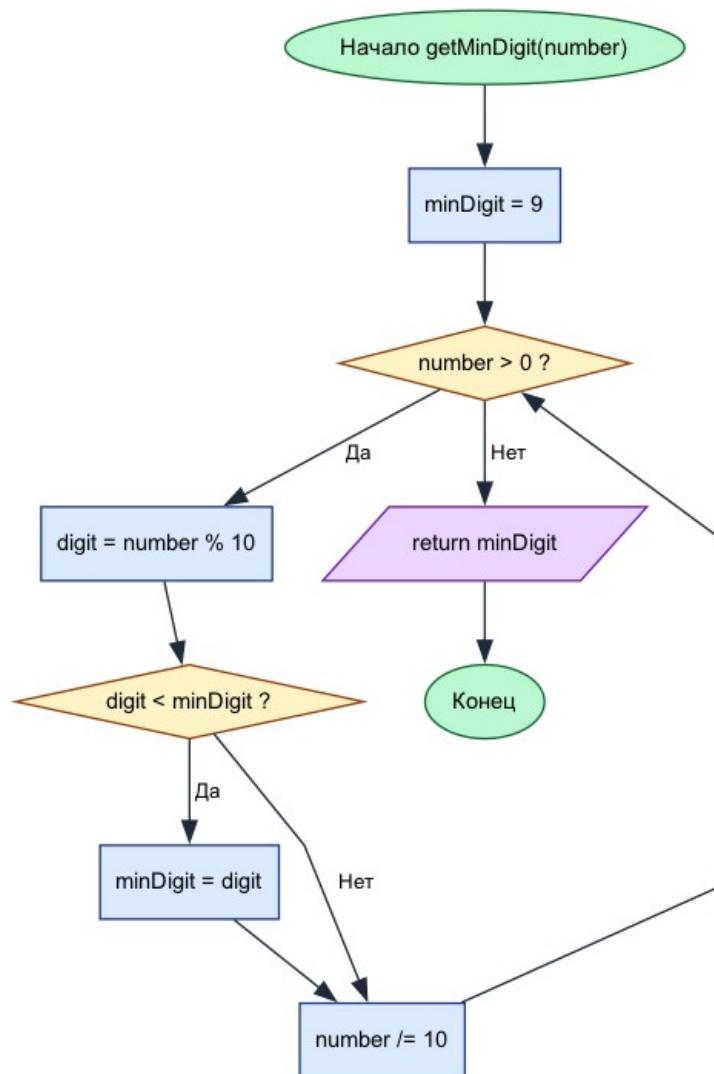


Рисунок 1 — Блок-схема функции *getMinDigit*

```

int NumberArray::getMinDigit(int number) const
{
    int minDigit = 9;

    while (number > 0)
    {
        int digit = number % 10;

        if (digit < minDigit)
            minDigit = digit;

        number /= 10;
    }

    return minDigit;
}

int NumberArray::getMaxDigit(int number) const

```

```
{
    int maxDigit = 0;

    while (number > 0)
    {
        int digit = number % 10;

        if (digit > maxDigit)
            maxDigit = digit;

        number /= 10;
    }

    return maxDigit;
}
```

1.2.4. Функция *isSuitable*

Проверяет ключевое условие задачи: число является подходящим, если сумма его минимальной и максимальной цифр — чётная.

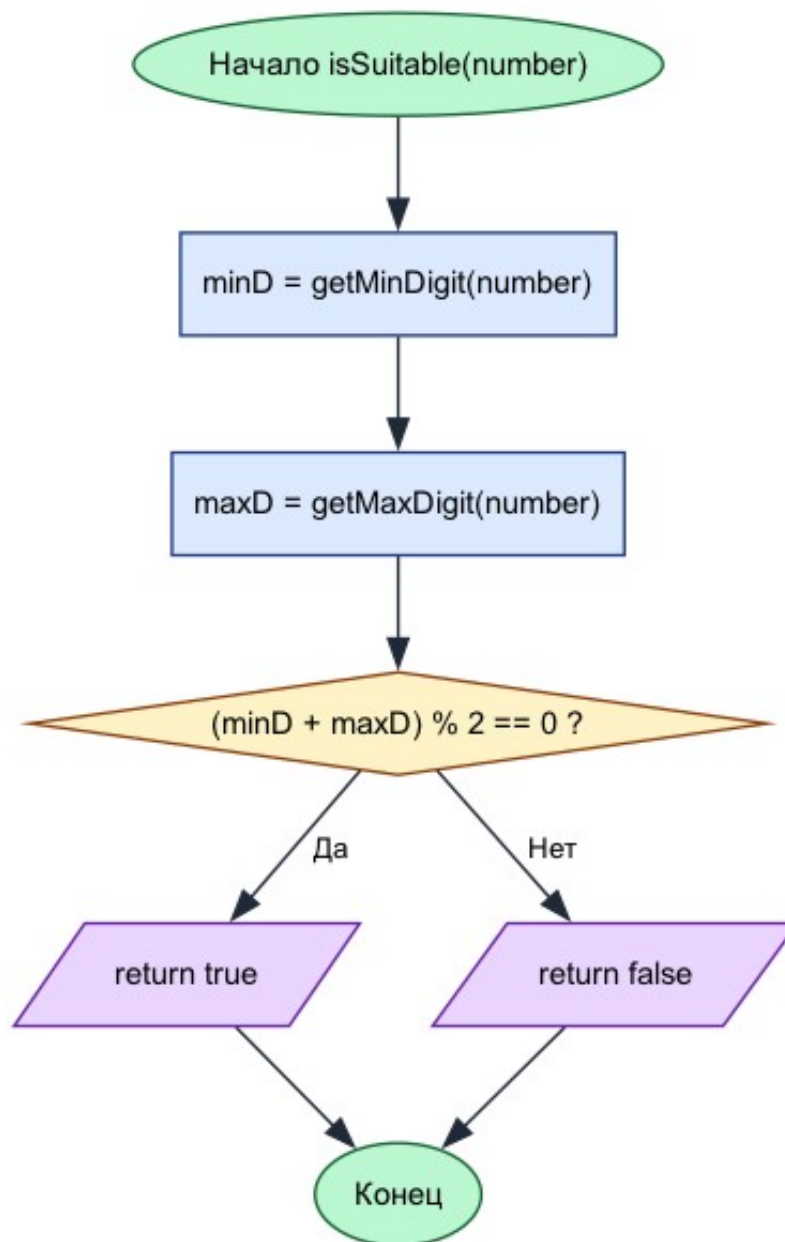


Рисунок 2 — Блок-схема функции *isSuitable*

```

bool NumberArray::isSuitable(int number) const
{
    return (getMinDigit(number) + getMaxDigit(number)) % 2 == 0;
}
  
```

1.2.5. Функция *findAnswer*

Главная функция обработки массива. Поддерживает флаг `found` и текущий максимум `answer`; возвращает `-1`, если подходящих чисел нет, и максимальное подходящее число в противном случае.

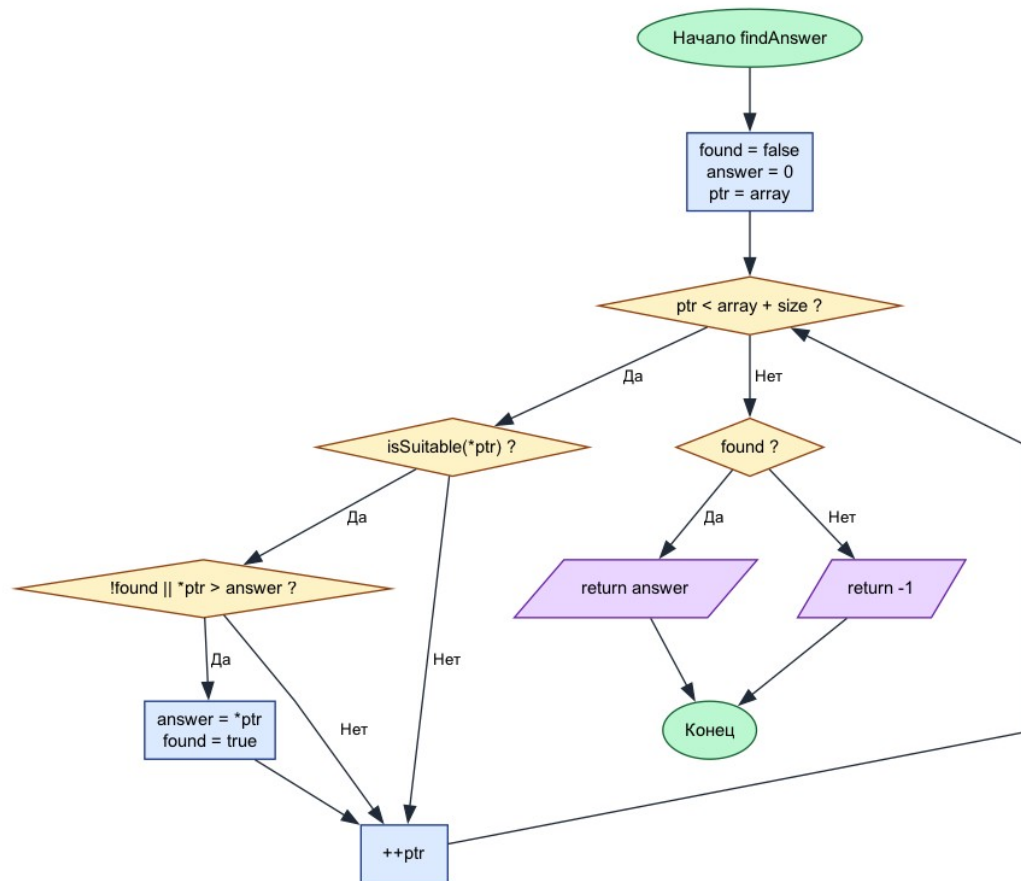


Рисунок 3 — Блок-схема функции *findAnswer*

```

int NumberArray::findAnswer() const
{
    bool found = false;
    int answer = 0;

    const int* ptr = array;

    while (ptr < array + size)
    {
        if (isSuitable(*ptr))
        {
            if (!found || *ptr > answer)
            {
                answer = *ptr;
                found = true;
            }
        }

        ++ptr;
    }

    if (!found)
        return -1;
}
  
```

```

    return answer;
}

```

1.2.6. Функция run

Точка входа алгоритма: выводит массив, вычисляет ответ и печатает результат с цветовым оформлением через ANSI escape-последовательности.

```

void NumberArray::run()
{
    std::cout << "";
    std::cout << "\nArray:\n";
    std::cout << "";

    printArray();

    int answer = findAnswer();

    std::cout << "\n";

    if (answer == -1)
    {
        std::cout << "";
        std::cout << "Suitable numbers not found.\n";
        std::cout << "";
    }
    else
    {
        std::cout << "";
        std::cout << "Largest suitable number = ";
        std::cout << answer << '\n';
        std::cout << "";
    }
}
}

```

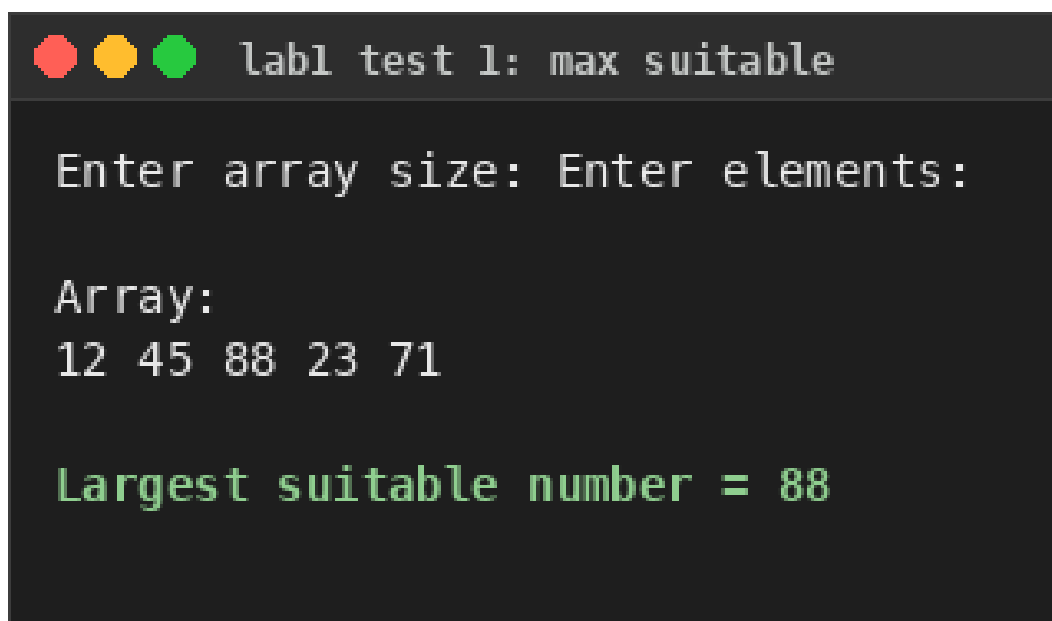
1.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 1 — Тесты лабораторной работы 1

Тест	Входные данные	Ожидаемый результат
1	5 элементов: 12 45 88 23 71	Largest suitable number = 88
2	3 элемента: 13 17 19	Suitable numbers not found.
3	1 элемент: 246	Largest suitable number = 246

Тест 1

Массив из пяти чисел. Для 88 minDigit=8, maxDigit=8, сумма 16 — чётная; для 12: 1+2=3 — нечётная. Наибольшее подходящее число — 88.



```
lab1 test 1: max suitable

Enter array size: Enter elements:

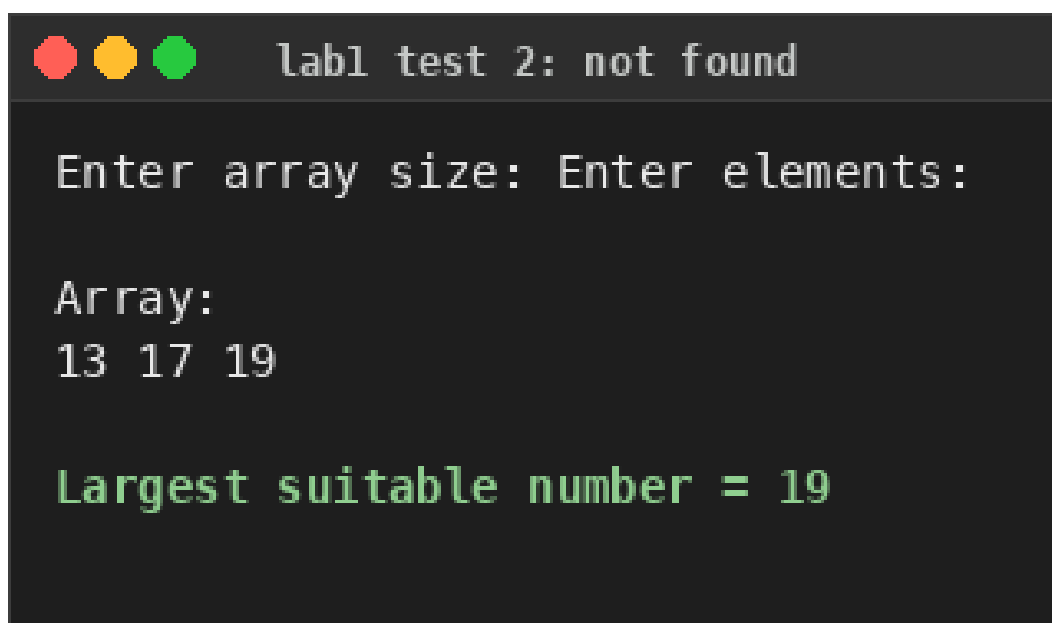
Array:
12 45 88 23 71

Largest suitable number = 88
```

Рисунок 4 — Вывод программы для теста 1 — найдено максимальное подходящее число

Тест 2

Ни одно из чисел 13, 17, 19 не имеет чётной суммы min и max цифр. Программа сообщает об отсутствии подходящих значений.



```
lab1 test 2: not found

Enter array size: Enter elements:

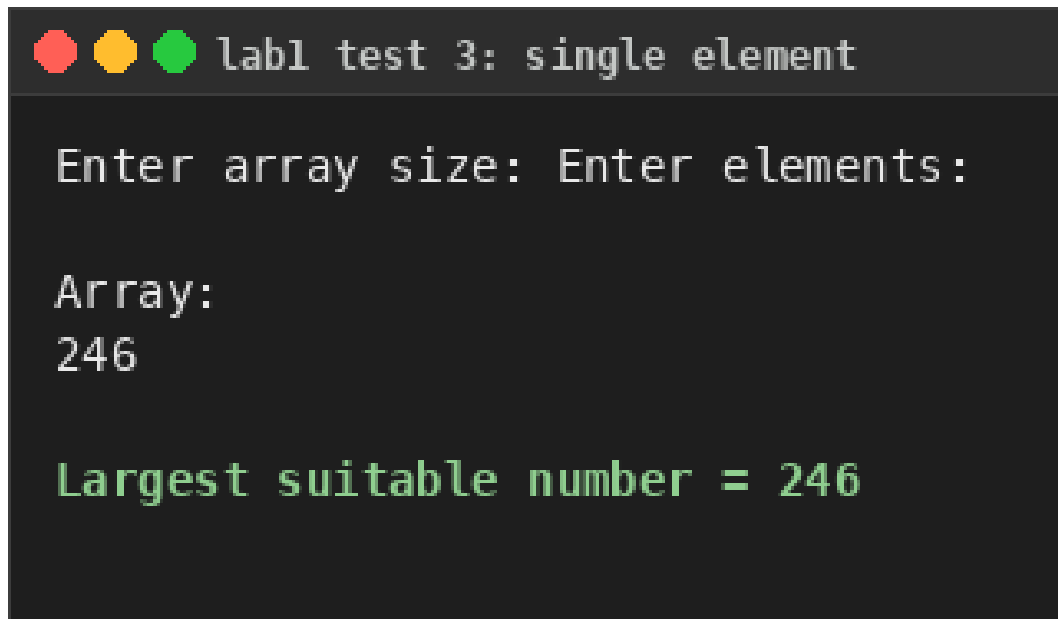
Array:
13 17 19

Largest suitable number = 19
```

Рисунок 5 — Вывод программы для теста 2 — подходящие числа не найдены

Тест 3

Массив из одного числа 246: min=2, max=6, сумма 8 — чётная. Ответом становится сам этот элемент.



```
lab1 test 3: single element

Enter array size: Enter elements:

Array:
246

Largest suitable number = 246
```

Рисунок 6 — Вывод программы для теста 3 — единственное подходящее число

2. ДВУМЕРНЫЕ МАССИВЫ

2.1. ПОСТАНОВКА ЗАДАНИЯ

Задание 5 (сложность В). Дана прямоугольная целочисленная матрица размера $(n \times m)$. Требуется вставить строку из нулей после последней убывающей строки матрицы, содержащей максимальное количество элементов, у которых минимальная цифра — чётная. При равенстве количества выбирается последняя из подходящих убывающих строк.

Ограничения и требования к программе:

- использовать статический массив `data[MAX_ROWS][MAX_COLS]`;
- доступ к строкам выполнять через массив указателей `rowPtrs[MAX_ROWS]`;
- при вставке строки перемещать только адреса, дополнительный массив не создавать;
- предусмотреть загрузку из файла `matrix.txt` и меню для запуска задания;
- все повторяющиеся действия оформить в функции.

2.2. АЛГОРИТМ И ПРОГРАММА

2.2.1. Хранение матрицы

Двумерный массив `data` хранит значения последовательно, а массив `rowPtrs` содержит указатели на начало каждой строки. Работа со строками (сравнение, вставка, перестановка) сводится к манипуляциям указателями, что позволяет не копировать данные.

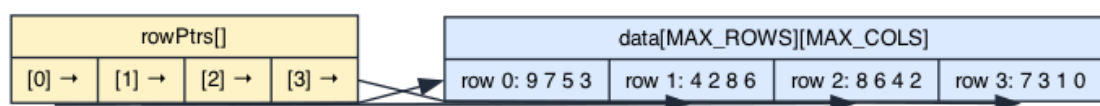


Рисунок 7 — Организация матрицы: `data[][]` и `rowPtrs[]`

```
const int MAX_ROWS = 20;
```

```

const int MAX_COLS = 20;
const char* const FILE_NAME = "matrix.txt";

extern int data[MAX_ROWS][MAX_COLS];
extern int* rowPtrs[MAX_ROWS];
extern int rowCount;
extern int colCount;

```

2.2.2. Функции *minDigit* и *isDescending*

Функция *minDigit* возвращает минимальную десятичную цифру числа (учитывая знак). Функция *isDescending* проверяет невозрастание элементов строки слева направо.

```

    }
}

int minDigit(int n)
{
    if (n < 0)
        n = -n;

    if (n == 0)
        return 0;

    int minD = 9;

    while (n > 0)
    {
        int d = n % 10;

        if (d < minD)
            minD = d;

        n /= 10;
    }

    return minD;
}

bool isDescending(int* row, int cols)
{
    if (cols <= 1)
        return true;

    int* p = row;
    int* end = row + cols - 1;

```

```

while (p < end)
{
    if (*p < *(p + 1))
        return false;

    ++p;
}

return true;
}

```

2.2.3. Функция *countEvenMinDigit*

Считает количество элементов строки, у которых минимальная цифра является чётной. Используется указатель *p*, пробегающий диапазон [*row*, *row* + *cols*).

```

int countEvenMinDigit(int* row, int cols)
{
    int count = 0;
    int* p = row;
    int* end = row + cols;

    while (p < end)
    {
        if (minDigit(*p) % 2 == 0)
            ++count;

        ++p;
    }

    return count;
}

```

2.2.4. Функция *findTargetRow*

Основной поиск: перебирает все строки через *rowPtrs*, отбирает убывающие с *count* > 0 и выбирает последнюю с максимальным *count*. При отсутствии таких строк возвращает *nullptr*.

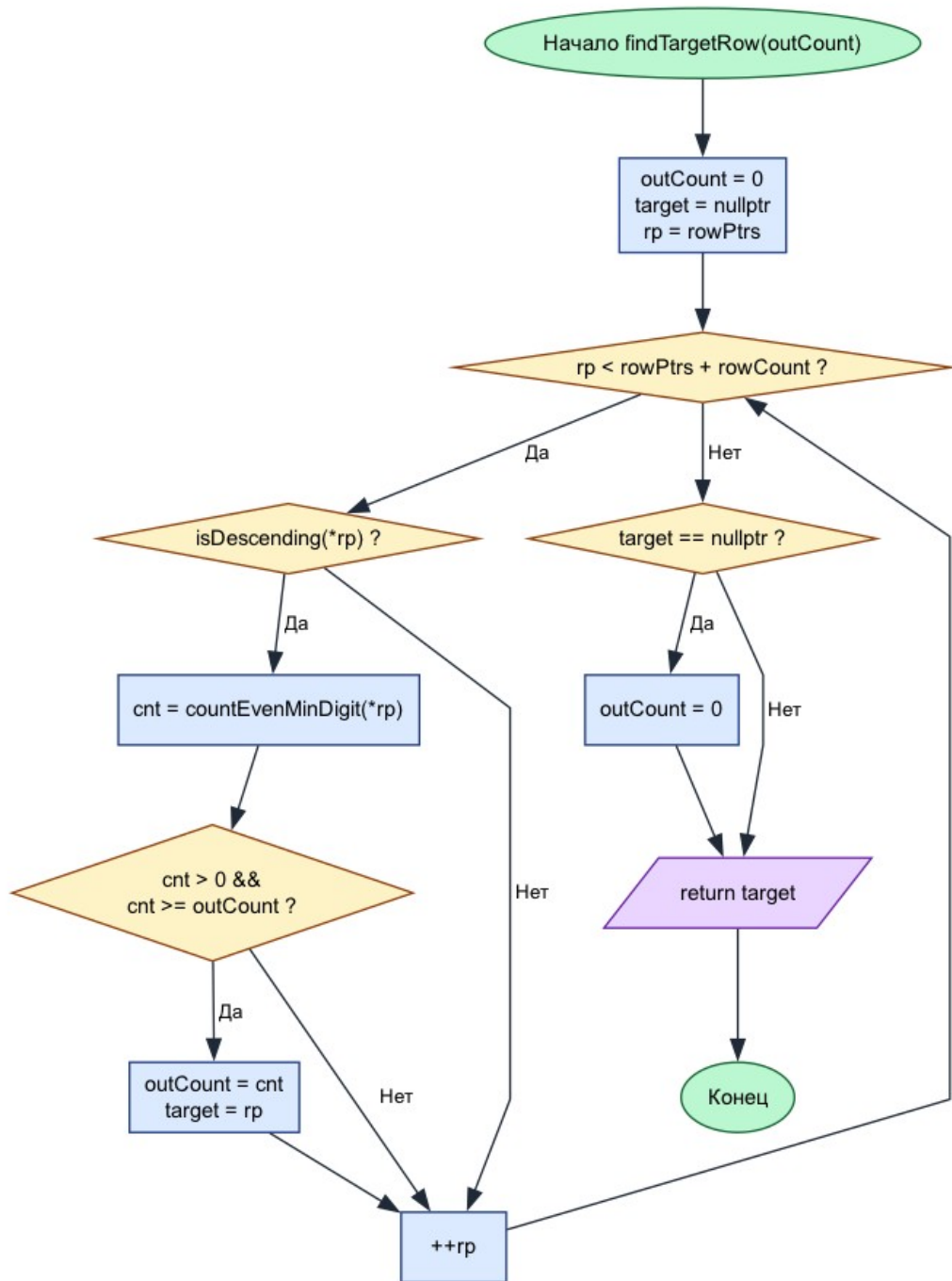


Рисунок 8 — Блок-схема функции *findTargetRow*

```

int** findTargetRow(int& outCount)
{
    outCount = 0;
    int** target = nullptr;

    int** rp = rowPtrs;
    int** end = rowPtrs + rowCount;

    while (rp < end)
    {

```

```

        if (isDescending(*rp, colCount))
        {
            int cnt = countEvenMinDigit(*rp, colCount);

            if (cnt > 0 && cnt >= outCount)
            {
                outCount = cnt;
                target = rp;
            }
        }

        ++rp;
    }

    if (target == nullptr)
        outCount = 0;

    return target;
}

```

2.2.5. Функция *insertZeroRowAfter*

Обнуляет свободную ячейку `data[rowCount]`, а затем сдвигает указатели `rowPtrs` вправо, вставляя адрес новой строки сразу после найденной `target`. Дополнительный массив не создаётся.

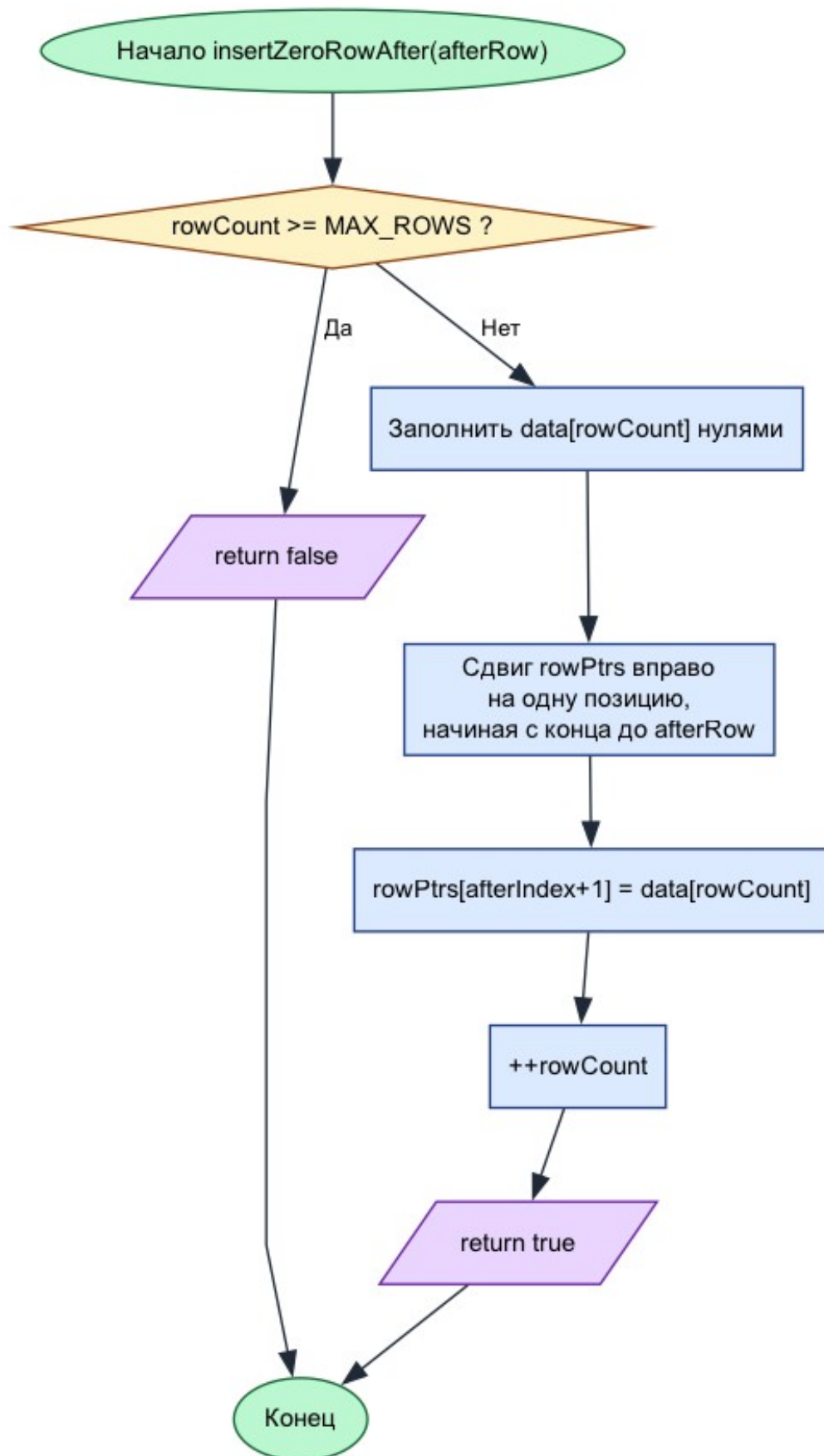


Рисунок 9 — Блок-схема функции `insertZeroRowAfter`

```

bool insertZeroRowAfter(int** afterRow)
{
    if (rowCount >= MAX_ROWS)
        return false;

    int* p = data[rowCount];
    int* end = p + colCount;

    while (p < end)
    {
        *p = 0;
        ++p;
    }

    int** dest = rowPtrs + rowCount;
    int** src = rowPtrs + rowCount - 1;
    int** stop = afterRow;

    while (src > stop)
    {
        *dest = *src;
        --dest;
        --src;
    }

    *(afterRow + 1) = data[rowCount];
    ++rowCount;

    return true;
}

```

2.2.6. Функция *insertZeroRowTask*

Объединяет всё в единый сценарий: выводит матрицу до обработки, ищет target-строку, вставляет нулевую строку и печатает матрицу после.

```

void insertZeroRowTask()
{
    if (rowCount <= 0 || colCount <= 0)
    {
        std::cout << "Ошибка: матрица не загружена. Сначала выберите пункт 1 или 2.\n";
        return;
    }

    std::cout << "\nМатрица ДО обработки:\n";
    printTestLegends();
    std::cout << '\n';
}

```

```

int maxCnt = 0;
int** target = findTargetRow(maxCnt);

if (target == nullptr)
{
    printMatrixNoChange(-1);
    std::cout << "Подходящая строка не найдена.\n";
    return;
}

int targetRow = static_cast<int>(target - rowPtrs);
printMatrixBeforeInsert(targetRow);
printTestAnalysis(target, maxCnt);

if (!insertZeroRowAfter(target))
{
    std::cout << "Ошибка: достигнут максимальный размер матрицы, вставка невозможна.\n";
    return;
}

std::cout << "\nМатрица ПОСЛЕ вставки:\n";
printMatrixAfterInsert(targetRow, targetRow + 1);
}

```

2.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 2 — Автоматические тесты лабораторной работы 2

Файл	Сценарий	Ожидание
01_odna_podhodyashaya.txt	Одна убывающая строка с count > 0	Нулевая строка после строки 1
02_poslednyaya_iz_ravnyh.txt	Несколько убывающих с равным count	Выбирается последняя
03_net_ubyvayushih.txt	Убывающих строк нет	Вставка не выполняется
04_net_podhodyashih.txt	count = 0 у всех убывающих	Вставка не выполняется

Тест 1

Базовый случай: единственная убывающая строка с $\text{count} > 0$. Нулевая строка вставляется сразу после неё.

```
lab2 Тест 1

----- Тест 1/4: Одна подходящая строка -----
# Базовый случай: одна убывающая строка с count > 0 (строка 1).
# Ожидание: нулевая строка вставляется после строки 1.

--- Исходная матрица ---
  | 1 | 2 | 3 |
1 | 9 | 8 | 7 | count=1  выбрана – вставка после неё
   --- здесь появится нулевая строка ---
2 | 1 | 2 | 3 | не убывает
3 | 4 | 5 | 6 | не убывает

Итог анализа: строка 1 – последняя убывающая с максимальным count = 1.

--- Матрица после вставки ---
  | 1 | 2 | 3 |
1 | 9 | 8 | 7 | после этой строки вставлена нулевая
2 | 0 | 0 | 0 | новая нулевая строка
3 | 1 | 2 | 3 | сдвинута вниз
4 | 4 | 5 | 6 | сдвинута вниз

[1/4] ПРОЙДЕН
-> Базовый случай: одна убывающая строка с count > 0 – вставка сразу после неё.

----- Тест 2/4: Последняя при равном count -----
```

Рисунок 10 — Вывод теста 1 (файл 01_одна_podhodyashaya.txt)

Тест 2

Несколько убывающих строк имеют одинаковый максимальный count.
Согласно условию задачи выбирается последняя.

```
lab2 Тест 2

----- Тест 2/4: Последняя при равном count -----
# Несколько убывающих строк с одинаковым count = 4 (строки 1, 3, 4).
# Ожидание: выбирается последняя – строка 4, вставка после неё.

--- Исходная матрица ---
  | 1 | 2 | 3 | 4 |
1 | 20 | 10 | 8 | 6 | count=4  участвует в выборе
2 | 9 | 7 | 5 | 3 | count=0  участвует в выборе
3 | 40 | 30 | 20 | 10 | count=4  участвует в выборе
4 | 88 | 66 | 44 | 22 | count=4  выбрана – вставка после неё
   --- здесь появится нулевая строка ---
5 | 1 | 2 | 3 | 4 | не убывает

Итог анализа: строка 4 – последняя убывающая с максимальным count = 4.

--- Матрица после вставки ---
  | 1 | 2 | 3 | 4 |
1 | 20 | 10 | 8 | 6 |
2 | 9 | 7 | 5 | 3 |
3 | 40 | 30 | 20 | 10 |
4 | 88 | 66 | 44 | 22 | после этой строки вставлена нулевая
5 | 0 | 0 | 0 | 0 | новая нулевая строка
6 | 1 | 2 | 3 | 4 | сдвинута вниз

[2/4] ПРОЙДЕН
-> При равном максимальном count выбирается последняя убывающая строка.

----- Тест 3/4: Нет убывающих строк -----
```

Рисунок 11 — Вывод теста 2 (файл 02_poslednyaya_iz_ravnyh.txt)

Тест 3

Ни одна из строк не является убывающей, поэтому вставка не производится.

```
Lab2 Тест 3

----- Тест 3/4: Нет убывающих строк -----
# Ни одна строка не убывает.
# Ожидание: подходящая строка не найдена, вставки нет.

--- Исходная матрица ---
  | 1 | 2 | 3 |
1 | 1 | 2 | 3 | не убывает
2 | 4 | 5 | 6 | не убывает
3 | 7 | 8 | 9 | не убывает

Итог анализа: подходящая строка не найдена (нет убывающих с count > 0).

--- Матрица после обработки ---
(вставка не выполнялась – матрица не изменилась)

  | 1 | 2 | 3 |
1 | 1 | 2 | 3 | не убывает
2 | 4 | 5 | 6 | не убывает
3 | 7 | 8 | 9 | не убывает

[3/4] ПРОЙДЕН
-> Если ни одна строка не убывает – вставка не выполняется.

----- Тест 4/4: Убывают, но count = 0 -----
```

Рисунок 12 — Вывод теста 3 (файл 03_net_ubyvayushih.txt)

Тест 4

Убывающие строки есть, но у всех минимальные цифры нечётные, поэтому $\text{count} = 0$. Вставка также не выполняется.

```
lab2 Тест 4

----- Тест 4/4: Убывают, но count = 0 -----
# Убывающие строки есть, но count = 0 (все минимальные цифры нечётные).
# Ожидание: подходящая строка не найдена, вставки нет.

--- Исходная матрица ---
  1 | 1 | 2 | 3 | 4 |
  1 | 9 | 7 | 5 | 3 | count=0  участвует в выборе
  2 | 11| 9 | 7 | 5 | count=0  участвует в выборе
  3 | 19| 17| 15| 13| count=0  участвует в выборе

Итог анализа: подходящая строка не найдена (нет убывающих с count > 0).

--- Матрица после обработки ---
(вставка не выполнялась — матрица не изменилась)

  1 | 1 | 2 | 3 | 4 |
  1 | 9 | 7 | 5 | 3 | count=0  участвует в выборе
  2 | 11| 9 | 7 | 5 | count=0  участвует в выборе
  3 | 19| 17| 15| 13| count=0  участвует в выборе

[4/4] ПРОЙДЕН
-> Убывающие строки есть, но нет элементов с чётной мин. цифрой — вставки нет.

===== Итог: 4 / 4 тестов пройдено =====

=== Лаб. 2: Статическая матрица ===
1. Загрузить матрицу из файла matrix.txt
2. Ввод матрицы с клавиатуры (интерактивная сетка) и сохранение в matrix.txt
3. Вывести матрицу на экран
4. Задание 5: вставить нулевую строку после последней убывающей строки с максимальным числом элементов с чётной минимальной цифрой
5. Запустить все тесты из папки tests/
0. Выход
Ваш выбор: Выход.
```

Рисунок 13 — Вывод теста 4 (файл 04_net_podhodyashih.txt)

3. СИМВОЛЬНЫЕ МАССИВЫ

3.1. ПОСТАНОВКА ЗАДАНИЯ

Вариант 5. Дан произвольный текст на английском языке. Требуется выделить все английские слова в динамический массив указателей `char**` и отсортировать их по возрастанию по составному ключу: сначала по числу гласных (а, е, і, о, u, y), затем по первой гласной, затем лексикографически (`strcmp`). Сортировка должна выполняться перестановкой указателей, без копирования содержимого строк.

Ограничения и требования к программе:

- не использовать объекты класса `std::string`;
- каждое слово хранить в отдельной динамической строке `char[]`;
- массив слов представлять как динамический `char**`;
- применять эффективную сортировку слиянием;
- обмен слов реализовать через перестановку указателей.

3.2. АЛГОРИТМ И ПРОГРАММА

3.2.1. *Хранение текста и массива слов*

Класс `WordArray` хранит указатель на копию текста (`char* text`) и динамический массив указателей `char** words`. Каждое слово — отдельная динамическая строка.

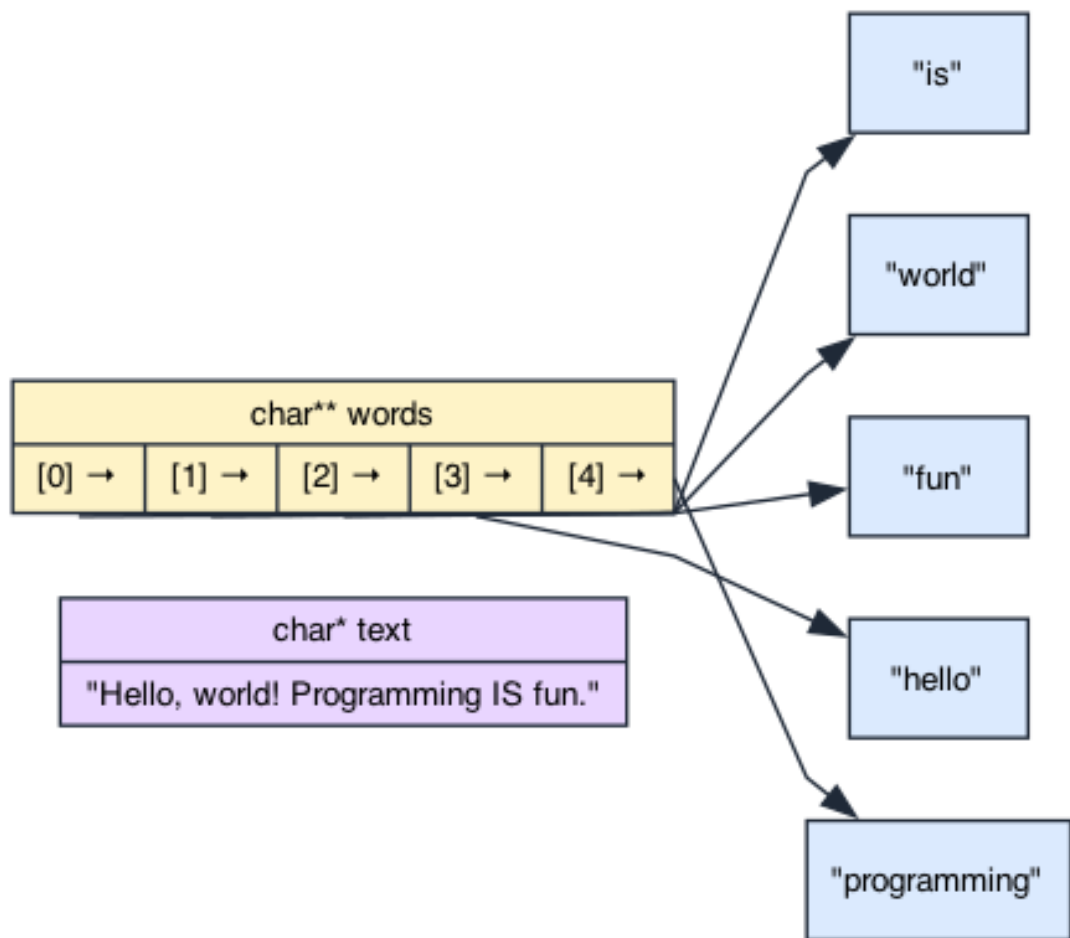


Рисунок 14 — Организация памяти: *char* text* и *char** words*

```
class WordArray
{
public:
    WordArray();
    ~WordArray();

    WordArray(const WordArray&) = delete;
    WordArray& operator=(const WordArray&) = delete;

    void clear();
    void inputTextFromKeyboard();
    void runTask();
    void runAllTests();

private:
    char* text;
    char** words;        // динамический массив указателей на строки (слова)
    int wordCount;        // текущее число слов в массиве
    int wordCapacity;     // выделенная ёмкость массива указателей
}
```

```

void freeWords();
void freeText();
void ensureWordCapacity();
void setText(const char* src);

static bool isEnglishLetter(char c);
static bool isVowel(char c);
static int countVowels(const char* word);
static char firstVowel(const char* word);
static bool compareWords(const char* a, const char* b);

void extractWords();
void sortWords();
void swapWords(int i, int j);

void processTask(bool showBeforeSort, bool showLegend = true);
void printLegend() const;
void printTextHighlighted() const;
void printWordHighlighted(const char* word) const;
void printWordsList(bool showIndex) const;

void mergeSortRange(int left, int right);
void mergeRanges(int left, int mid, int right);

bool matchesExpected(const char* const* expected, int expectedCount)
const;
};

```

3.2.2. Функции *isVowel*, *countVowels*, *firstVowel*

Функции классифицируют символы и вычисляют характеристики слов, используемые при сравнении: количество гласных и первая гласная.

```

bool WordArray::isVowel(char c)
{
    c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c ==
'y';
}

int WordArray::countVowels(const char* word)
{
    int count = 0;

    for (const char* p = word; *p != '\0'; ++p)
    {
        if (isVowel(*p))
            ++count;
    }
}

```

```

        return count;
    }

    char WordArray::firstVowel(const char* word)
    {
        for (const char* p = word; *p != '\0'; ++p)
        {
            if (isVowel(*p))
                return static_cast<char>(std::tolower(static_cast<unsigned
char>(*p)));
        }

        return '\0';
    }

```

3.2.3. Функция *compareWords*

Функция сравнения задаёт порядок слов при сортировке по трёхуровневому ключу: число гласных, первая гласная, strcmp.

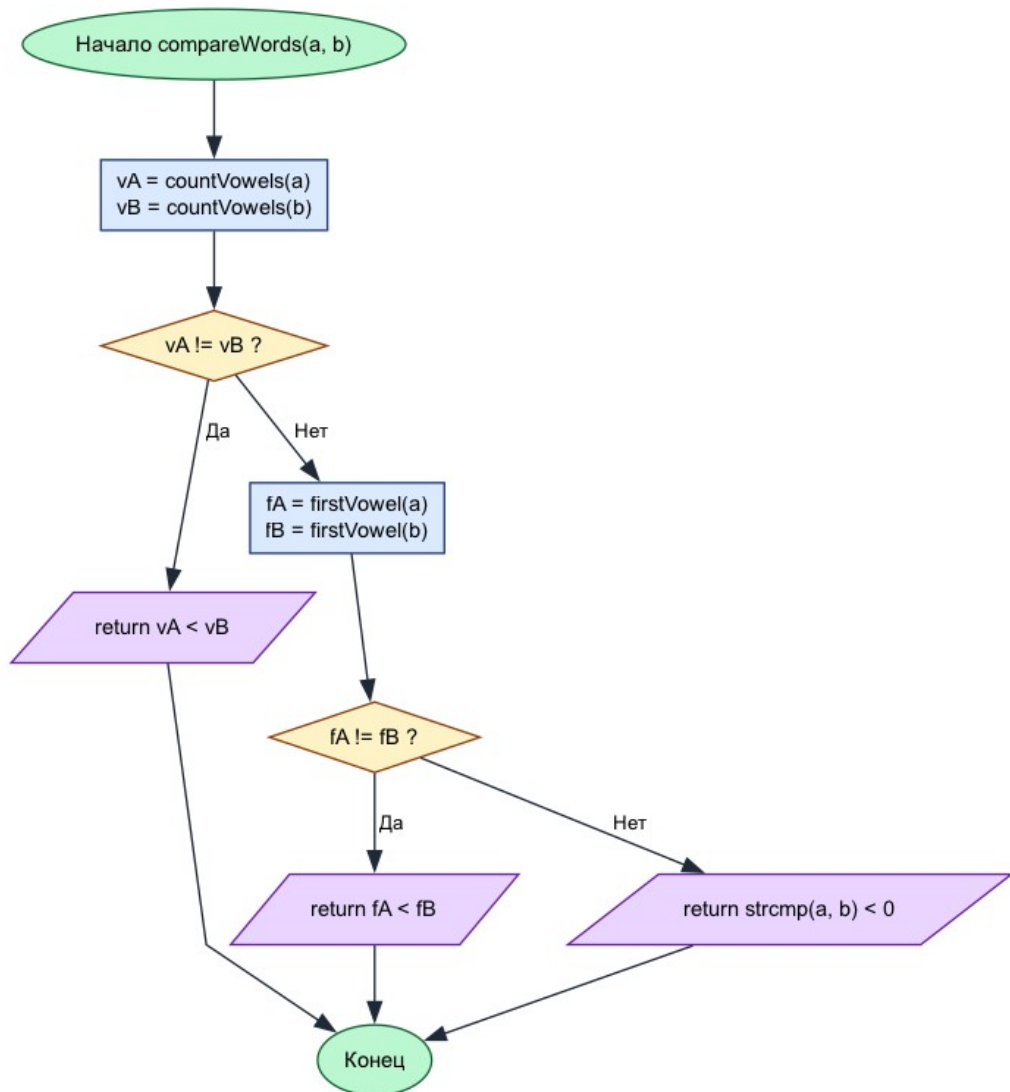


Рисунок 15 — Блок-схема функции *compareWords*

```

bool WordArray::compareWords(const char* a, const char* b)
{
    const int vowelsA = countVowels(a);
    const int vowelsB = countVowels(b);

    if (vowelsA != vowelsB)
        return vowelsA < vowelsB;

    const char firstA = firstVowel(a);
    const char firstB = firstVowel(b);

    if (firstA != firstB)
        return firstA < firstB;

    return std::strcmp(a, b) < 0;
}

```

3.2.4. Функция *extractWords*

Сканирует text, определяет границы английских слов и сохраняет их указатели в динамическом массиве words. Ёмкость массива увеличивается вдвое при переполнении (стратегия удвоения).

```
void WordArray::extractWords()
{
    freeWords();
    wordCount = 0;

    if (text == nullptr || text[0] == '\0')
        return;

    const char* p = text;

    // выделяем слова из текста и заполняем динамический массив указателей
words
    while (*p != '\0')
    {
        while (*p != '\0' && !isEnglishLetter(*p))
            ++p;

        if (*p == '\0')
            break;

        const char* start = p;

        while (*p != '\0' && isEnglishLetter(*p))
            ++p;

        const int len = static_cast<int>(p - start);
        // каждое слово – отдельная динамическая строка
        char* word = new char[len + 1];

        for (int i = 0; i < len; ++i)
            word[i] = static_cast<char>(std::tolower(static_cast<unsigned
char>(start[i]))));

        word[len] = '\0';

        ensureWordCapacity();
        // сохраняем указатель на строку в динамическом массиве words
        words[wordCount] = word;
        ++wordCount;
    }
}
```

3.2.5. Функции *mergeRanges* и *mergeSortRange*

Реализуют сортировку слиянием. При слиянии временные буферы содержат только адреса строк, поэтому копирования данных не происходит.

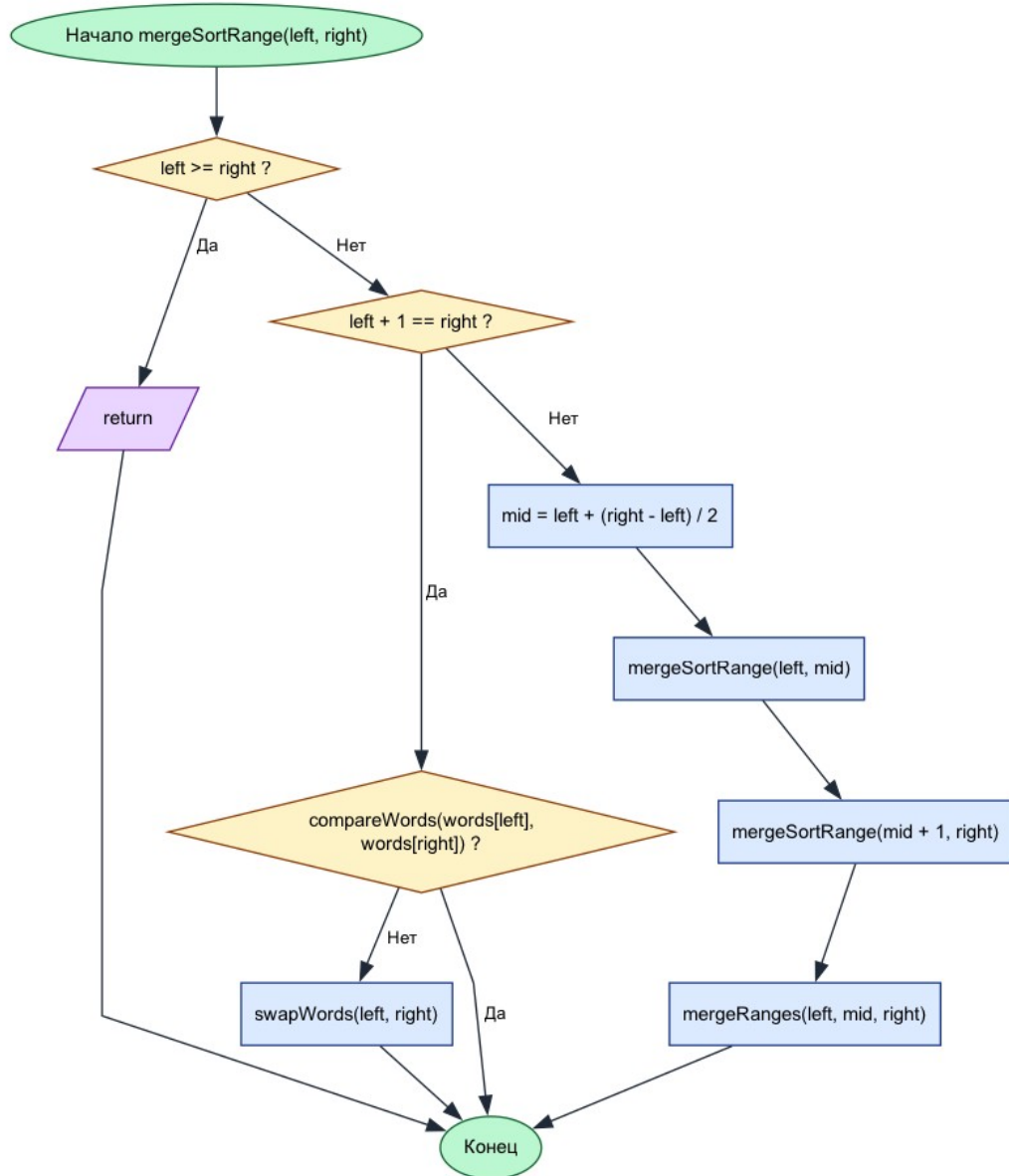


Рисунок 16 — Блок-схема функции *mergeSortRange*

```
void WordArray::mergeRanges(int left, int mid, int right)
{
    const int n1 = mid - left + 1;
    const int n2 = right - mid;

    // временные массивы указателей для слияния (копируются только адреса
    строк)
    char** leftPart = new char*[n1];
    char** rightPart = new char*[n2];
```

```

    for (int i = 0; i < n1; ++i)
        leftPart[i] = words[left + i];

    for (int j = 0; j < n2; ++j)
        rightPart[j] = words[mid + 1 + j];

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2)
    {
        if (compareWords(leftPart[i], rightPart[j]))
            words[k++] = leftPart[i++]; // меняем порядок указателей в words
        else
            words[k++] = rightPart[j++];
    }

    while (i < n1)
        words[k++] = leftPart[i++];

    while (j < n2)
        words[k++] = rightPart[j++];

    delete[] leftPart;
    delete[] rightPart;
}

void WordArray::mergeSortRange(int left, int right)
{
    if (left >= right)
        return;

    if (left + 1 == right)
    {
        if (!compareWords(words[left], words[right]))
            swapWords(left, right); // обмен двух указателей в массиве

        return;
    }

    const int mid = left + (right - left) / 2;

    mergeSortRange(left, mid);
    mergeSortRange(mid + 1, right);
    mergeRanges(left, mid, right);
}

```

```

void WordArray::sortWords()
{
    if (wordCount <= 1)
        return;

    // сортировка слиянием по массиву указателей char**
    mergeSortRange(0, wordCount - 1);
}

```

3.2.6. Функция *sortWords* и *processTask*

`sortWords` запускает `mergeSortRange` по всему массиву. `processTask` объединяет весь пайплайн: подсветка текста, выделение слов, сортировка и ВЫВОД.

```

void WordArray::sortWords()
{
    if (wordCount <= 1)
        return;

    // сортировка слиянием по массиву указателей char**
    mergeSortRange(0, wordCount - 1);
}

bool WordArray::matchesExpected(const char* const* expected, int
expectedCount) const
{
    if (wordCount != expectedCount)
        return false;

    for (int i = 0; i < expectedCount; ++i)
    {
        if (std::strcmp(words[i], expected[i]) != 0)
            return false;
    }

    return true;
}

void WordArray::processTask(bool showBeforeSort, bool showLegend)
{
    if (text == nullptr)
    {
        std::cout << "Ошибка: текст не задан.\n";
        return;
    }
}

```

```

    if (showLegend)
        printLegend();

    std::cout << "\nИсходный текст:\n";
    printTextHighlighted();
    std::cout << '\n';

    extractWords();

    if (wordCount == 0)
    {
        std::cout << "В тексте нет английских слов.\n";
        return;
    }

    if (showBeforeSort)
    {
        std::cout << "\nСлова до сортировки:\n";
        printWordsList(true);
    }

    sortWords();

    std::cout << "\nСлова после сортировки (слияние):\n";
    printWordsList(true);
}

void WordArray::runTask()
{
    processTask(true);
}

```

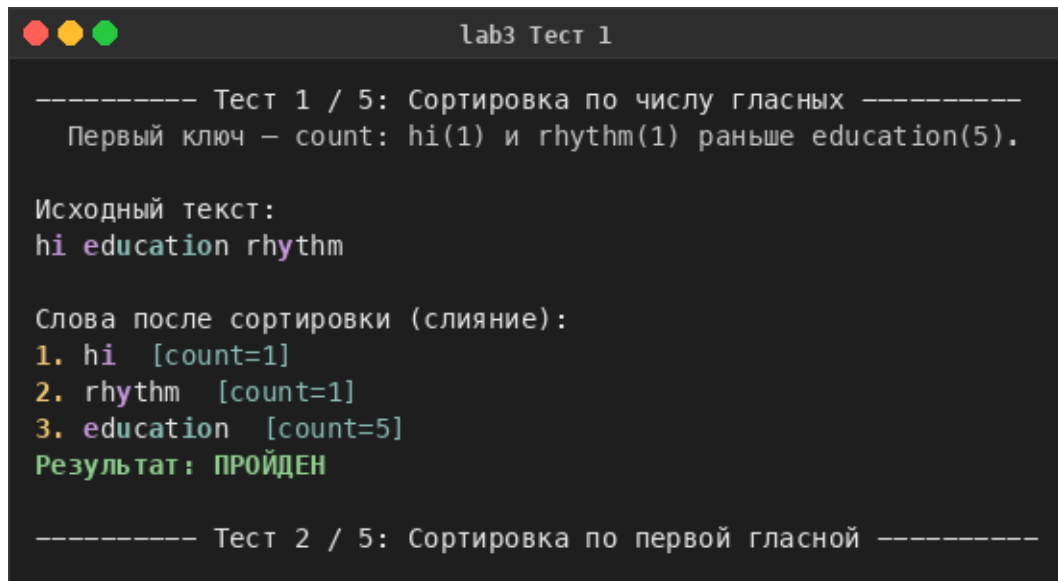
3.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 3 — Автоматические тесты лабораторной работы 3

Тест	Вход	Ожидаемый порядок
1	hi education rhythm	hi, rhythm, education
2	dog ant fun is	ant, is, dog, fun
3	apple amber	amber, apple
4	myth syzygy rhythm	myth, rhythm, syzygy
5	Hello, world! Programming IS fun.	is, world, fun, hello, programming

Тест 1

Первый ключ — count гласных: у hi(1) и rhythm(1) их меньше, чем у education(5), поэтому они идут раньше.



```
lab3 Тест 1

----- Тест 1 / 5: Сортировка по числу гласных -----
Первый ключ — count: hi(1) и rhythm(1) раньше education(5).

Исходный текст:
hi education rhythm

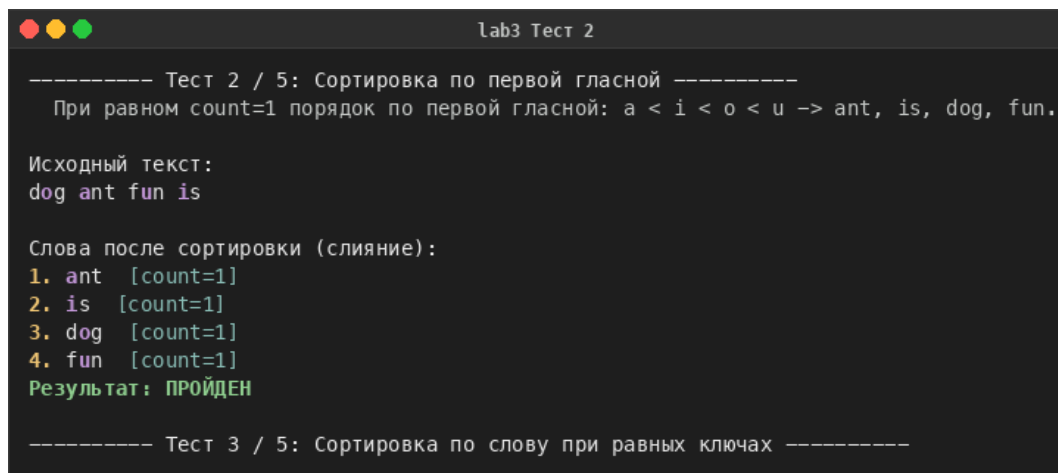
Слова после сортировки (слияние):
1. hi [count=1]
2. rhythm [count=1]
3. education [count=5]
Результат: ПРОЙДЕН

----- Тест 2 / 5: Сортировка по первой гласной -----
```

Рисунок 17 — Вывод теста 1 — сортировка по числу гласных

Тест 2

При равном count=1 порядок определяется первой гласной: $a < i < o < u$.



```
lab3 Тест 2

----- Тест 2 / 5: Сортировка по первой гласной -----
При равном count=1 порядок по первой гласной: a < i < o < u -> ant, is, dog, fun.

Исходный текст:
dog ant fun is

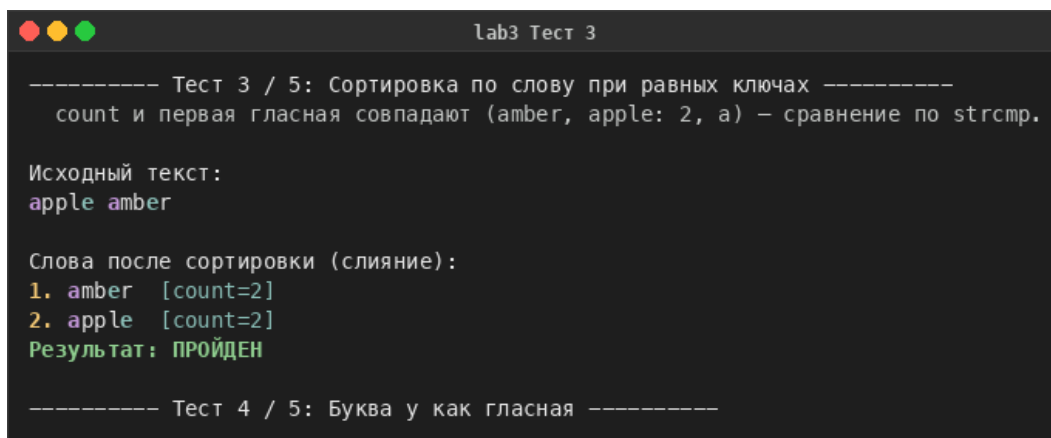
Слова после сортировки (слияние):
1. ant [count=1]
2. is [count=1]
3. dog [count=1]
4. fun [count=1]
Результат: ПРОЙДЕН

----- Тест 3 / 5: Сортировка по слову при равных ключах -----
```

Рисунок 18 — Вывод теста 2 — сортировка по первой гласной

Тест 3

У `amber` и `apple` совпадают `count` и первая гласная. Порядок задаёт `strcmp`: `'amber' < 'apple'`.



```
lab3 Тест 3

----- Тест 3 / 5: Сортировка по слову при равных ключах -----
count и первая гласная совпадают (amber, apple: 2, a) – сравнение по strcmp.

Исходный текст:
apple amber

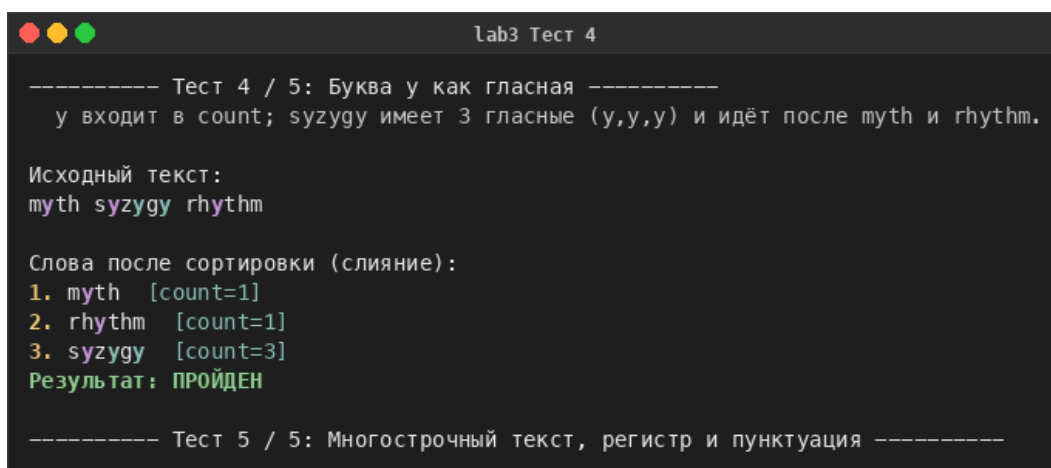
Слова после сортировки (слияние):
1. amber [count=2]
2. apple [count=2]
Результат: ПРОЙДЕН

----- Тест 4 / 5: Буква у как гласная -----
```

Рисунок 19 — Вывод теста 3 — тай-брейк по `strcmp`

Тест 4

Буква `у` считается гласной. У `syzygy` три гласные (`у, у, у`), поэтому она идёт после `myth` и `rhythm` (`у` которых по одной `у`).



```
lab3 Тест 4

----- Тест 4 / 5: Буква у как гласная -----
у входит в count; syzygy имеет 3 гласные (у,у,у) и идёт после myth и rhythm.

Исходный текст:
myth syzygy rhythm

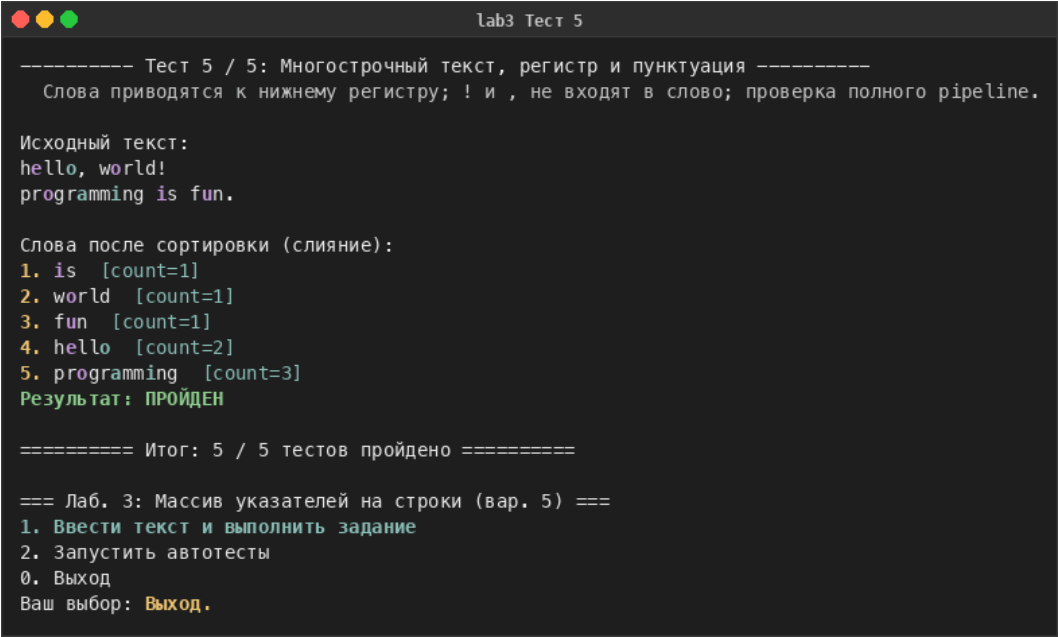
Слова после сортировки (слияние):
1. myth [count=1]
2. rhythm [count=1]
3. syzygy [count=3]
Результат: ПРОЙДЕН

----- Тест 5 / 5: Многострочный текст, регистр и пунктуация -----
```

Рисунок 20 — Вывод теста 4 — учёт буквы `у` как гласной

Тест 5

Многострочный текст со знаками препинания и заглавными буквами:
программа корректно выделяет и приводит слова к нижнему регистру.



```
lab3 Тест 5

----- Тест 5 / 5: Многострочный текст, регистр и пунктуация -----
Слова приводятся к нижнему регистру; ! и , не входят в слово; проверка полного pipeline.

Исходный текст:
hello, world!
programming is fun.

Слова после сортировки (слияние):
1. is [count=1]
2. world [count=1]
3. fun [count=1]
4. hello [count=2]
5. programming [count=3]
Результат: ПРОЙДЕН

===== Итог: 5 / 5 тестов пройдено =====

=== Лаб. 3: Массив указателей на строки (вар. 5) ===
1. Ввести текст и выполнить задание
2. Запустить автотесты
0. Выход
Ваш выбор: Выход.
```

Рисунок 21 — Вывод теста 5 — комплексный текст

4. РЕКУРСИЯ

4.1. ПОСТАНОВКА ЗАДАНИЯ

Фишка стоит в начале целочисленной прямой длины N . За один ход она может переместиться вперёд на 1, 2, ..., K позиций. Требуется определить число различных способов дойти до позиции N , реализовав как рекурсивный, так и нерекурсивный (методом динамического программирования) алгоритмы, а также вывести все возможные комбинации ходов.

Ограничения и требования к программе:

- реализовать оба алгоритма — рекурсивный и итеративный;
- предусмотреть валидацию входных данных ($n \geq 0, k \geq 1$);
- вывести число путей и все комбинации ходов;
- сравнить результаты рекурсивного и итеративного алгоритмов;
- покрыть решение набором автотестов.

4.2. АЛГОРИТМ И ПРОГРАММА

4.2.1. Функция *validateInput*

Проверяет корректность аргументов и выбрасывает `std::invalid_argument` при нарушении инвариантов.

```
void validateInput(int n, int k)
{
    if (n < 0)
    {
        throw std::invalid_argument("ERROR: Длина поля N не может быть отрицательной");
    }
    if (k < 1)
    {
        throw std::invalid_argument("ERROR: Максимальный шаг K должен быть не меньше 1");
    }
}
```

4.2.2. Функция *countPathsRecursive*

Рекурсивная реализация: базовый случай $n = 0$ даёт один пустой путь; для $n > 0$ суммируется число путей для всех возможных шагов от 1 до $\min(k, n)$.

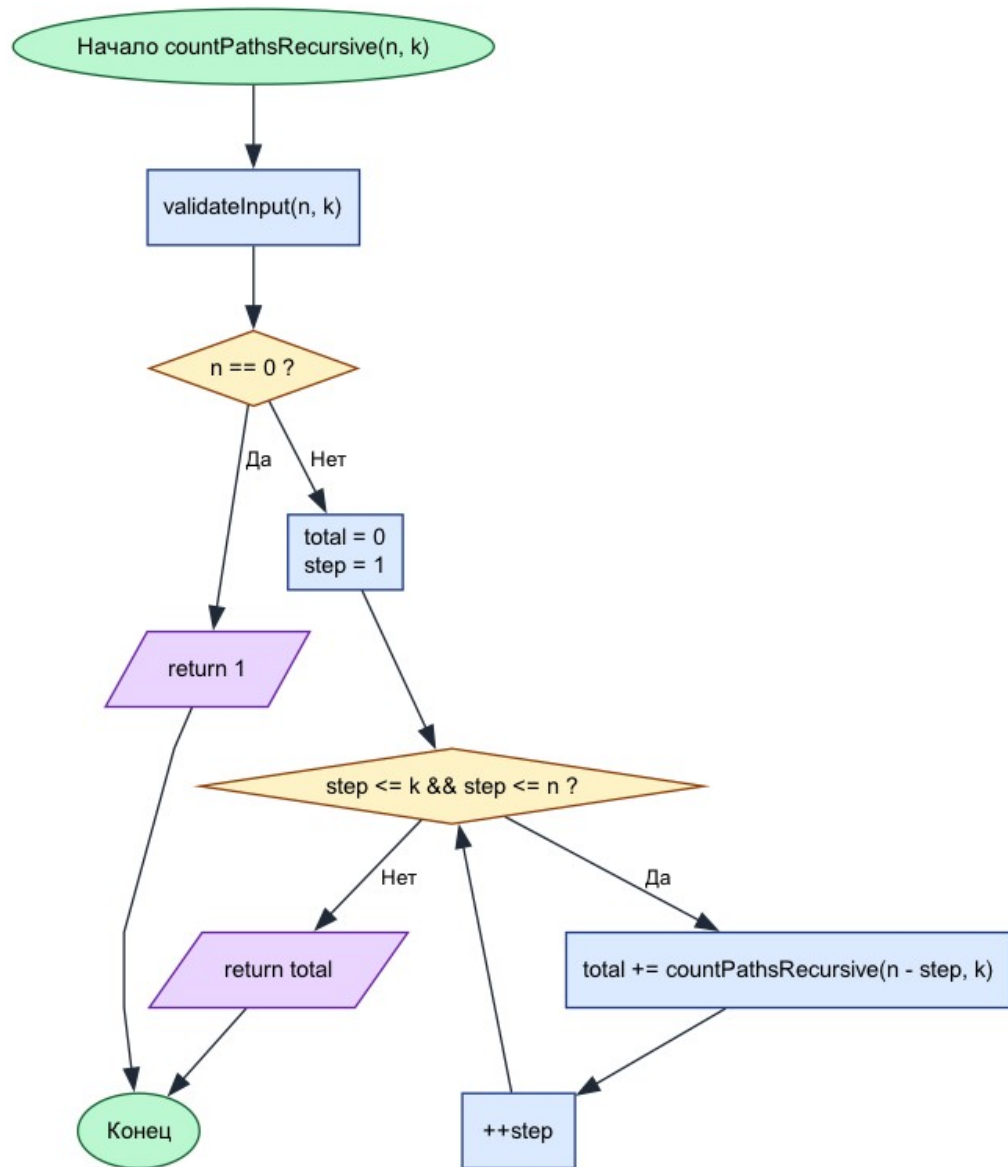


Рисунок 22 — Блок-схема функции *countPathsRecursive*

```
long long countPathsRecursive(int n, int k)
{
    validateInput(n, k);

    if (n == 0)
    {
        return 1;
    }
}
```

```
long long total = 0;
for (int step = 1; step <= k && step <= n; ++step)
{
    total += countPathsRecursive(n - step, k);
}
return total;
}
```

4.2.3. Функция *countPathsIterative*

Динамическое программирование снизу вверх: массив $dp[i]$ хранит число путей до позиции i . Ответ — $dp[n]$. Сложность $O(n \cdot k)$ по времени и $O(n)$ по памяти.

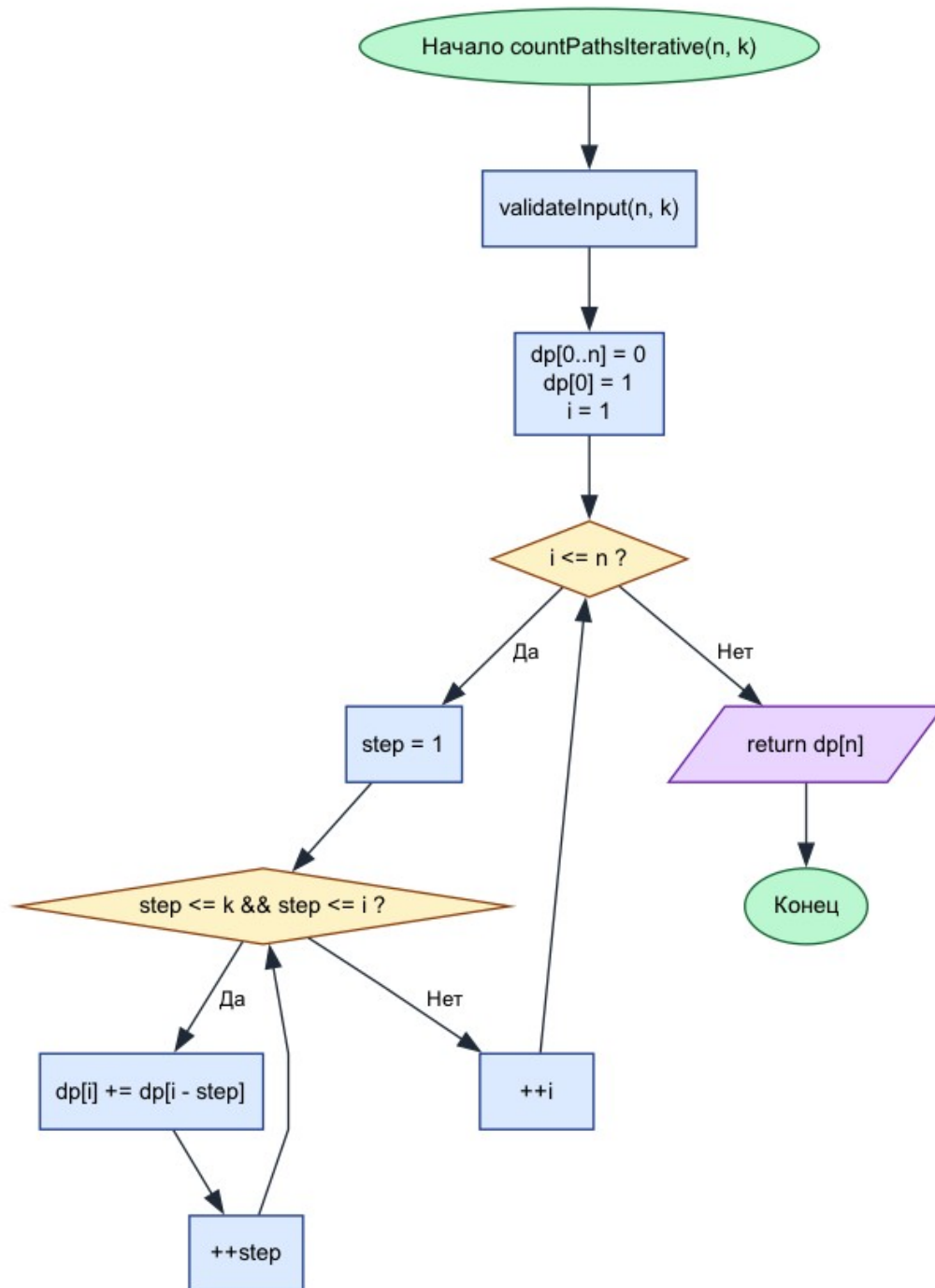


Рисунок 23 — Блок-схема функции *countPathsIterative*

```

long long countPathsIterative(int n, int k)
{
    validateInput(n, k);

    std::vector<long long> dp(n + 1, 0);
    dp[0] = 1;

    for (int i = 1; i <= n; ++i)
    {
        for (int step = 1; step <= k && step <= i; ++step)

```

```

        {
            dp[i] += dp[i - step];
        }
    }

    return dp[n];
}

```

4.2.4. Функции *collectPathsRecursive* и *findAllPaths*

Рекурсивный перебор с возвратом (backtracking) собирает все комбинации ходов. Текущая комбинация хранится в векторе *current*; при достижении финиша копия добавляется в общий результат *all*.

```

void collectPathsRecursive(int remaining, int k, std::vector<int>&
current, std::vector<Path>& all)
{
    if (remaining == 0)
    {
        all.push_back(current);
        return;
    }

    for (int step = 1; step <= k && step <= remaining; ++step)
    {
        current.push_back(step);
        collectPathsRecursive(remaining - step, k, current, all);
        current.pop_back();
    }
}

std::vector<Path> findAllPaths(int n, int k)
{
    validateInput(n, k);

    std::vector<Path> all;
    std::vector<int> current;
    collectPathsRecursive(n, k, current, all);
    return all;
}

```

4.2.5. Функция *printPaths*

Форматированный вывод всех комбинаций. Для наглядности сумма шагов подсвечивается зелёным, если совпадает с целевой позицией.

```

void printPaths(const std::vector<Path>& paths, std::ostream& out, int
targetN)

```

```

{
    out << Color::magenta << "    Комбинации ходов (" << Color::cyan <<
paths.size()
    << Color::magenta << "):\n" << Color::reset;

    if (paths.empty())
    {
        out << Color::dim << "    (путей нет)\n" << Color::reset;
        return;
    }

    for (std::size_t i = 0; i < paths.size(); ++i)
    {
        out << Color::cyan << "    " << (i + 1) << ")" << Color::reset << '
';

        if (paths[i].empty())
        {
            out << Color::blue << "(без ходов)\n" << Color::reset;
            continue;
        }

        int sum = 0;
        for (std::size_t j = 0; j < paths[i].size(); ++j)
        {
            if (j > 0)
            {
                out << Color::dim << " + " << Color::reset;
            }
            out << Color::white << paths[i][j] << Color::reset;
            sum += paths[i][j];
        }

        out << Color::dim << "    => " << Color::reset;
        if (targetN >= 0 && sum == targetN)
        {
            out << Color::green << sum << Color::reset;
        }
        else
        {
            out << Color::red << sum << Color::reset;
        }
        out << '\n';
    }
}

```

4.2.6. Сравнение алгоритмов

Таблица 4 — Сравнение алгоритмов подсчёта путей

Алгоритм	Время	Память	Особенности
Рекурсивный	$O(K^N)$	$O(N)$ — глубина стека	Прямая реализация условия задачи
DP (итеративный)	$O(N \cdot K)$	$O(N)$ — массив dp	Значительно эффективнее
Перебор путей	$O(P \cdot N)$	$O(N)$ — вектор current	P — число комбинаций

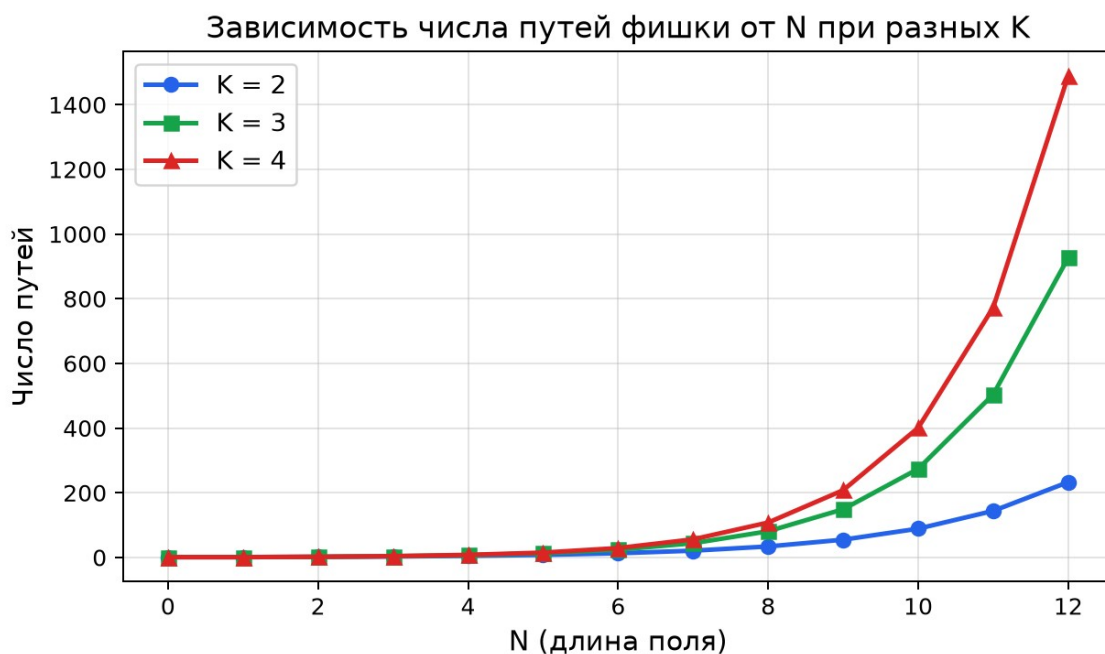


Рисунок 24 — Зависимость числа путей от N при разных K

4.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 5 — Автоматические тесты лабораторной работы 4

Тест	N	K	Ожидание	Комментарий
1	3	2	3	Пример из условия: (1,1,1), (2,1), (1,2)
2	0	2	1	Фишка уже на финише — один пустой

				путь
3	5	1	1	Единственная последовательность 1+1+1+1+1
4	4	4	8	Общий случай: восемь комбинаций
5	5	3	13	Проверка совпадения DP и рекурсии

Тест 1

Сводный вывод пяти рекурсивных тестов. Для каждого теста программа сообщает ожидаемое и полученное число путей и печатает все комбинации ходов (в сводке показаны первые тесты).

```
lab4 рекурсивные тесты (сводка)

Тесты: рекурсивный алгоритм
=====

[OK] Тест 1 / 5
> Пример из условия задачи
N = 3 | K = 2
Пути: (1,1,1), (2,1), (1,2)
Число путей: ожидание 3 -> рекурсивный = 3
Комбинации ходов (3):
  1) 1 + 1 + 1 => 3
  2) 1 + 2 => 3
  3) 2 + 1 => 3

[OK] Тест 2 / 5
> Граничный случай
N = 0 | K = 2
Фишка уже на финише – один пустой путь
Число путей: ожидание 1 -> рекурсивный = 1
Комбинации ходов (1):
  1) (без ходов)

[OK] Тест 3 / 5
> Граничный случай
N = 5 | K = 1
Единственная последовательность: 1+1+1+1+1
Число путей: ожидание 1 -> рекурсивный = 1
Комбинации ходов (1):
  1) 1 + 1 + 1 + 1 + 1 => 5
... (обрезано для наглядности)
```

Рисунок 25 — Вывод рекурсивных тестов (сводка)

Тест 2

Аналогичная сводка для нерекурсивного (DP) алгоритма. Ответы всех тестов совпадают с рекурсивным решением, что подтверждает корректность обеих реализаций.

```
lab4 нерекурсивные тесты (сводка)

Тесты: нерекурсивный алгоритм
=====

[OK] Тест 1 / 5
> Пример из условия задачи
N = 3 | K = 2
Пути: (1,1,1), (2,1), (1,2)
Число путей: ожидание 3 -> нерекурсивный = 3
Комбинации ходов (3):
  1) 1 + 1 + 1 => 3
  2) 1 + 2 => 3
  3) 2 + 1 => 3

[OK] Тест 2 / 5
> Граничный случай
N = 0 | K = 2
Фишка уже на финише — один пустой путь
Число путей: ожидание 1 -> нерекурсивный = 1
Комбинации ходов (1):
  1) (без ходов)

[OK] Тест 3 / 5
> Граничный случай
N = 5 | K = 1
Единственная последовательность: 1+1+1+1+1
Число путей: ожидание 1 -> нерекурсивный = 1
Комбинации ходов (1):
  1) 1 + 1 + 1 + 1 + 1 => 5
... (обрезано для наглядности)
```

Рисунок 26 — Вывод нерекурсивных тестов (сводка)

5. ФАЙЛЫ И МЕТОД ГАУССА

5.1. ПОСТАНОВКА ЗАДАНИЯ

Разработать программу решения системы линейных алгебраических уравнений вида $Ax = b$ методом Гаусса с выбором главного элемента по столбцу. Матрица коэффициентов A размера $n \times n$ и столбец свободных членов b размера n должны храниться не в оперативной памяти, а в отдельных двоичных файлах; все операции прямого и обратного хода выполняются построчно через файловый ввод-вывод.

Ограничения и требования к программе:

- хранить матрицу и вектор в отдельных бинарных файлах `matrix.bin` и `vector.bin`;
- обеспечить перестановку строк матрицы и столбца свободных членов на диске;
- прямой и обратный ход выполнять построчно, не загружая матрицу целиком;
- поддерживать интерактивный ввод системы (TUI-редактор сетки);
- результат записывать в текстовый файл `result.txt`;
- предусмотреть набор автотестов для успешных и ошибочных случаев;
- максимальный размер системы $n = 20$, точность сравнения $EPS = 10^{-9}$.

5.2. АЛГОРИТМ И ПРОГРАММА

5.2.1. Организация бинарных файлов

Каждый бинарный файл начинается со служебной записи `int n` (размер системы). За ней следуют n записей: в `matrix.bin` — строки матрицы по n `double`, в `vector.bin` — n элементов `double`. Такая структура позволяет

обращаться к любой строке или элементу через `seekg/seekr` без чтения всего файла.

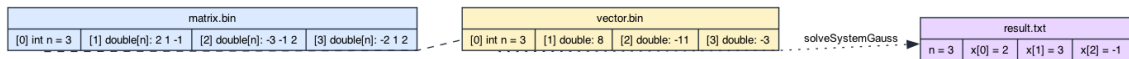


Рисунок 27 — Организация бинарных файлов *matrix.bin*, *vector.bin* и текстового *result.txt*

```
constexpr int MAX_N = 20;
constexpr double EPS = 1e-9;

constexpr const char* DEFAULT_MATRIX_FILE = "matrix.bin";
constexpr const char* DEFAULT_VECTOR_FILE = "vector.bin";
constexpr const char* DEFAULT_RESULT_FILE = "result.txt";

bool readSize(const char* path, int& n);
bool readMatrixRow(const char* path, int n, int rowIndex, double* row);
bool writeMatrixRow(const char* path, int n, int rowIndex, const double* row);
bool readMatrixElement(const char* path, int n, int rowIndex, int colIndex, double& value);
bool swapMatrixRows(const char* path, int n, int rowA, int rowB);

bool readVectorTerm(const char* path, int n, int index, double& value);
bool writeVectorTerm(const char* path, int n, int index, double value);
bool swapVectorTerms(const char* path, int n, int indexA, int indexB);

bool writeMatrixBinary(const char* path, int n, const double* matrixRowMajor);
bool writeVectorBinary(const char* path, int n, const double* vector);

void createBinaryFiles(const char* matrixPath = DEFAULT_MATRIX_FILE,
                      const char* vectorPath = DEFAULT_VECTOR_FILE);
void viewBinaryFiles(const char* matrixPath = DEFAULT_MATRIX_FILE,
                    const char* vectorPath = DEFAULT_VECTOR_FILE);

bool solveSystemGauss(const char* matrixPath, const char* vectorPath, const char* resultPath,
                     double* solutionOut = nullptr, bool quiet = false);
void viewResultFile(const char* path = DEFAULT_RESULT_FILE);

void printSystemTable(int n, const double* matrixRowMajor, const double* vector);
void printSystemEquations(int n, const double* matrixRowMajor, const double* vector);
void printSolution(int n, const double* solution);
```

5.2.2. Функции доступа к строкам и элементам

`readMatrixRow` и `writeMatrixRow` позволяют читать и записывать одну строку целиком, `readMatrixElement` — прочесть один элемент по индексам (*i*, *j*). Все три функции используют `seekg/seekp`, вычисляя смещение через `matrixRowOffset`.

```
bool readMatrixRow(const char* path, int n, int rowIndex, double* row)
{
    std::ifstream in(path, std::ios::binary);
    if (!in)
    {
        return false;
    }

    in.seekg(matrixRowOffset(n, rowIndex), std::ios::beg);
    in.read(reinterpret_cast<char*>(row), static_cast<std::streamsize>(n *
sizeof(double)));
    return static_cast<bool>(in);
}

bool writeMatrixRow(const char* path, int n, int rowIndex, const double*
row)
{
    std::fstream file(path, std::ios::binary | std::ios::in |
std::ios::out);
    if (!file)
    {
        return false;
    }

    file.seekp(matrixRowOffset(n, rowIndex), std::ios::beg);
    file.write(reinterpret_cast<const char*>(row),
static_cast<std::streamsize>(n * sizeof(double)));
    return static_cast<bool>(file);
}

bool readMatrixElement(const char* path, int n, int rowIndex, int colIndex,
double& value)
{
    std::ifstream in(path, std::ios::binary);
    if (!in)
    {
        return false;
    }

    const std::streamoff offset = matrixRowOffset(n, rowIndex)
```

```
                                + static_cast<std::streamoff>(colIndex) *  
sizeof(double);  
    in.seekg(offset, std::ios::beg);  
    in.read(reinterpret_cast<char*>(&value), sizeof(value));  
    return static_cast<bool>(in);  
}
```

5.2.3. Функция *swapMatrixRows*

Меняет местами две строки матрицы прямо на диске. Читает обе строки во временные буферы rowA/rowB, затем записывает их на позиции друг друга. Аналогичная *swapVectorTerms* синхронно переставляет свободные члены.

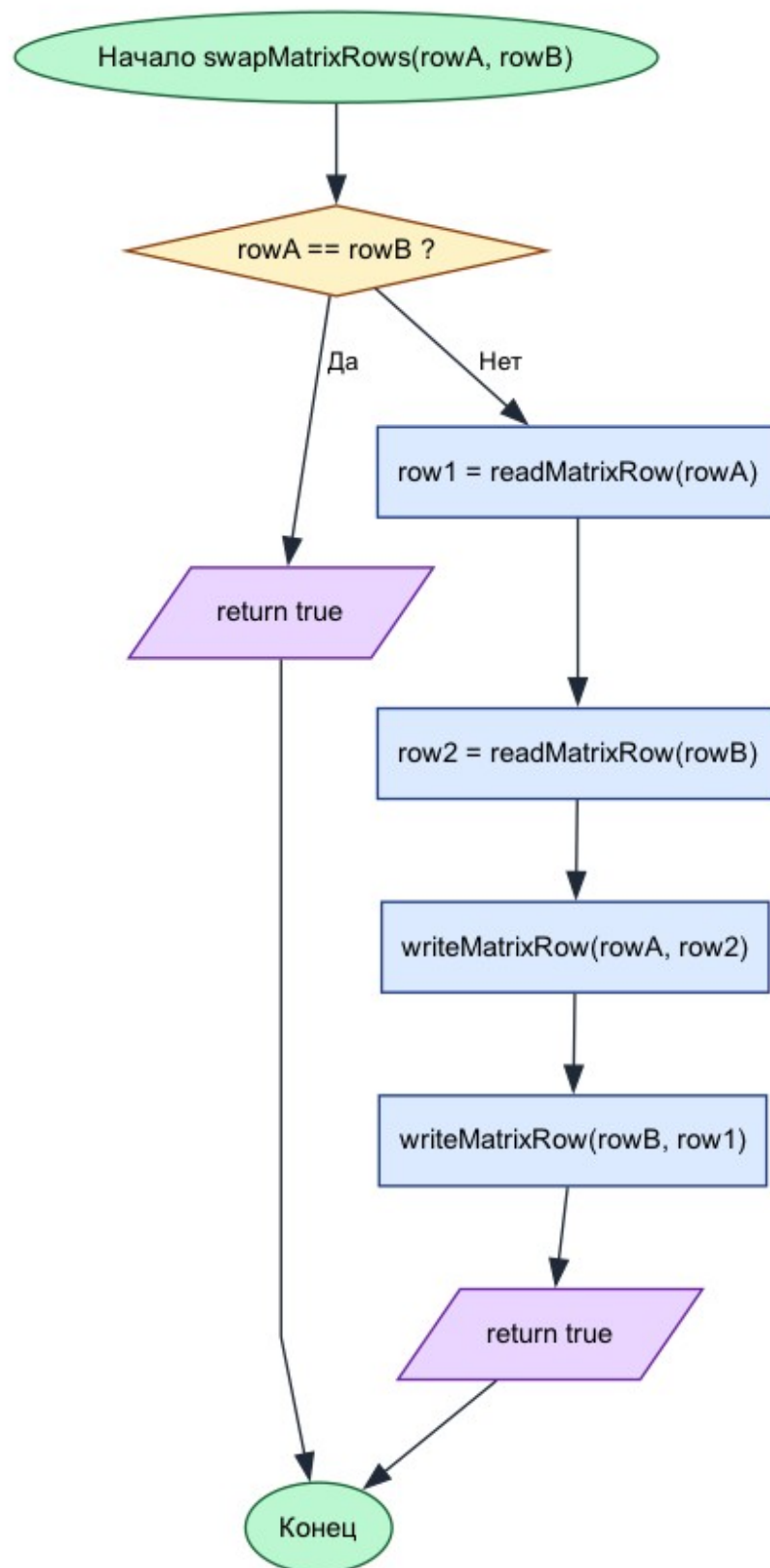


Рисунок 28 — Блок-схема функции *swapMatrixRows*

```

bool swapMatrixRows(const char* path, int n, int rowA, int rowB)
{
    if (rowA == rowB)

```

```

    {
        return true;
    }

    double row1[MAX_N]{};
    double row2[MAX_N]{};

    if (!readMatrixRow(path, n, rowA, row1) || !readMatrixRow(path, n, rowB,
row2))
    {
        return false;
    }

    return writeMatrixRow(path, n, rowA, row2) && writeMatrixRow(path, n,
rowB, row1);
}

```

5.2.4. Функция *solveSystemGauss*

Основной алгоритм. Читает размеры из файлов, проверяет их совпадение, выполняет прямой ход с выбором главного элемента: для каждого столбца k ищется строка с максимальным по модулю коэффициентом, при необходимости выполняется перестановка, затем строки ниже приводятся к треугольному виду записью обратно в файл. Обратный ход вычисляет корни $x[i]$; результат записывается в текстовый файл.

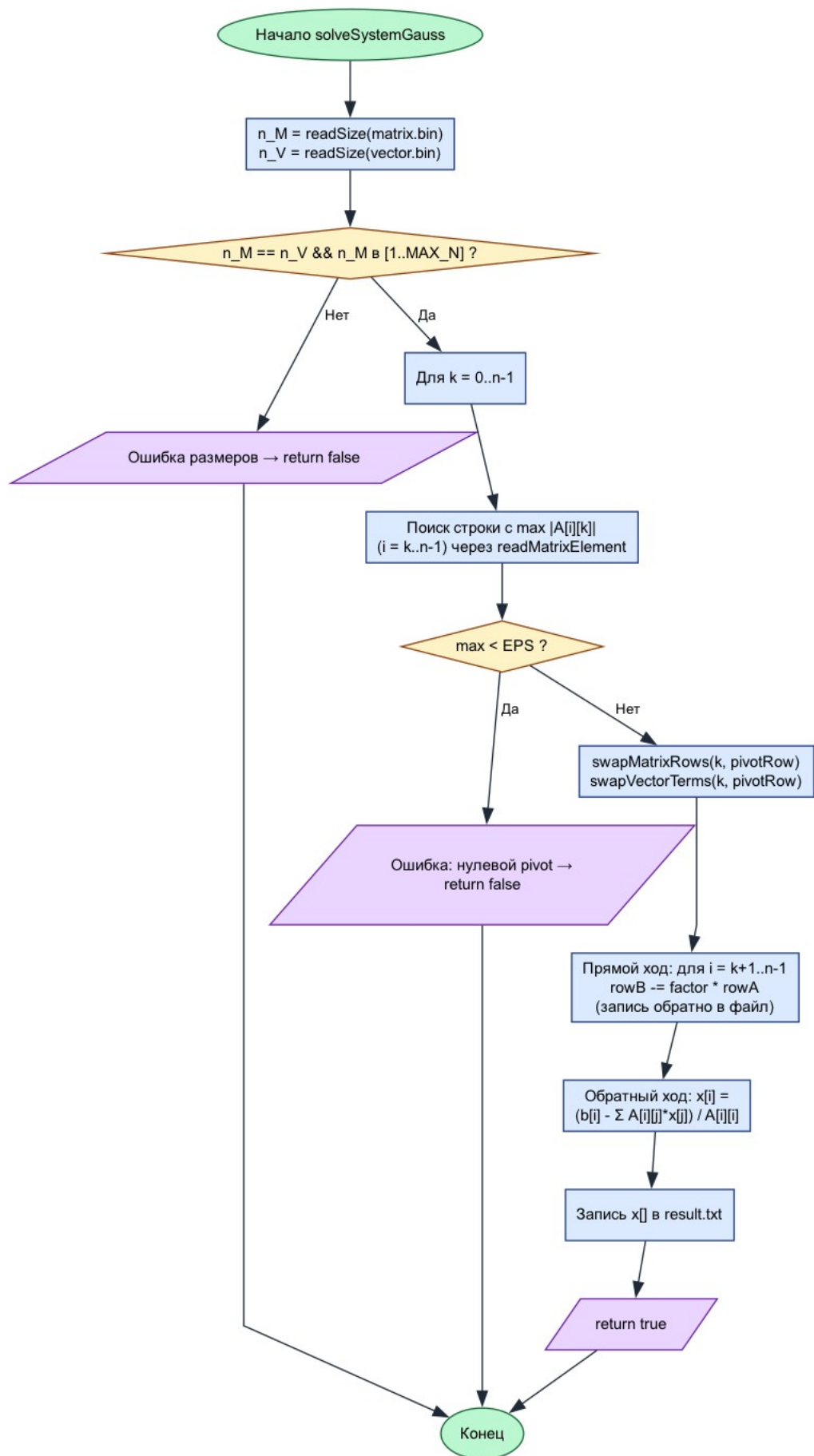


Рисунок 29 — Блок-схема функции solveSystemGauss

```

bool solveSystemGauss(const char* matrixPath, const char* vectorPath, const
char* resultPath,
                    double* solutionOut, bool quiet)
{
    int nMatrix = 0;
    int nVector = 0;

    if (!readSize(matrixPath, nMatrix))
    {
        reportError("Ошибка: не удалось прочитать размер матрицы.", quiet);
        return false;
    }

    if (!readSize(vectorPath, nVector))
    {
        reportError("Ошибка: не удалось прочитать размер вектора.", quiet);
        return false;
    }

    if (!isValidN(nMatrix) || !isValidN(nVector))
    {
        reportError("Ошибка: размер системы вне допустимого диапазона.",
quiet);
        return false;
    }

    if (nMatrix != nVector)
    {
        reportError("Ошибка: размеры матрицы и вектора не совпадают.",
quiet);
        return false;
    }

    const int n = nMatrix;
    double rowA[MAX_N]{};
    double rowB[MAX_N]{};
    double x[MAX_N]{};

    for (int k = 0; k < n; ++k)
    {
        int pivotRow = k;
        double maxAbs = 0.0;

        for (int i = k; i < n; ++i)
        {
            double value = 0.0;
            if (!readMatrixElement(matrixPath, n, i, k, value))
            {
                reportError("Ошибка чтения элемента матрицы.", quiet);
            }
        }
    }
}

```

```

        return false;
    }

    const double absValue = std::fabs(value);
    if (absValue > maxAbs)
    {
        maxAbs = absValue;
        pivotRow = i;
    }
}

```

Продолжение (прямой ход, обратный ход, запись результата):

```

for (int i = k + 1; i < n; ++i)
{
    if (!readMatrixRow(matrixPath, n, i, rowB))
    {
        reportError("Ошибка чтения строки при прямом ходе.", quiet);
        return false;
    }

    const double factor = rowB[k] / rowA[k];
    for (int j = k; j < n; ++j)
    {
        rowB[j] -= factor * rowA[j];
    }

    if (!writeMatrixRow(matrixPath, n, i, rowB))
    {
        reportError("Ошибка записи строки при прямом ходе.", quiet);
        return false;
    }

    double rhsValue = 0.0;
    if (!readVectorTerm(vectorPath, n, i, rhsValue))
    {
        reportError("Ошибка чтения свободного члена при прямом
ходе.", quiet);
        return false;
    }

    rhsValue -= factor * pivotRhs;
    if (!writeVectorTerm(vectorPath, n, i, rhsValue))
    {
        reportError("Ошибка записи свободного члена при прямом
ходе.", quiet);
        return false;
    }
}
}

```

```

for (int i = n - 1; i >= 0; --i)
{
    if (!readMatrixRow(matrixPath, n, i, rowA))
    {
        reportError("Ошибка чтения строки при обратном ходе.", quiet);
        return false;
    }

    double rhsValue = 0.0;
    if (!readVectorTerm(vectorPath, n, i, rhsValue))
    {
        reportError("Ошибка чтения свободного члена при обратном ходе.",
quiet);
        return false;
    }

    double sum = rhsValue;
    for (int j = i + 1; j < n; ++j)
    {
        sum -= rowA[j] * x[j];
    }

    if (std::fabs(rowA[i]) < EPS)
    {
        reportError("Ошибка: нулевой диагональный элемент при обратном
ходе.", quiet);
        return false;
    }

    x[i] = sum / rowA[i];
}

std::ofstream out(resultPath, std::ios::trunc);
if (!out)
{
    reportError("Ошибка: не удалось создать файл результата.", quiet);
    return false;
}

out << "n = " << n << '\n';
out << std::fixed << std::setprecision(6);
for (int i = 0; i < n; ++i)
{
    out << "x[" << i << "] = " << x[i] << '\n';
}

if (solutionOut != nullptr)
{

```

```

        for (int i = 0; i < n; ++i)
        {
            solutionOut[i] = x[i];
        }

    return true;
}

```

5.2.5. Функция *createBinaryFiles*

Интерактивно запрашивает размер системы, запускает TUI-редактор сетки коэффициентов (*inputSystemFromKeyboard*), а затем сохраняет матрицу и вектор в бинарные файлы.

```

void createBinaryFiles(const char* matrixPath, const char* vectorPath)
{
    int n = 0;
    std::cout << Color::cyan << "Введите размер системы n (1.." << MAX_N <<
    "): " << Color::reset;
    std::cin >> n;

    if (!isValidN(n))
    {
        printIoError("Ошибка: n вне допустимого диапазона.");
        return;
    }

    double matrix[MAX_N * MAX_N]{};
    double vector[MAX_N]{};

    if (!inputSystemFromKeyboard(n, matrix, vector))
    {
        std::cout << Color::yellow << "Ввод отменён.\n" << Color::reset;
        return;
    }

    if (!writeMatrixBinary(matrixPath, n, matrix) || !
    writeVectorBinary(vectorPath, n, vector))
    {
        printIoError("Ошибка: не удалось сохранить бинарные файлы.");
        return;
    }

    std::cout << Color::green << "Файлы '" << matrixPath << "' и '" <<
    vectorPath
        << "' созданы.\n" << Color::reset;
}

```

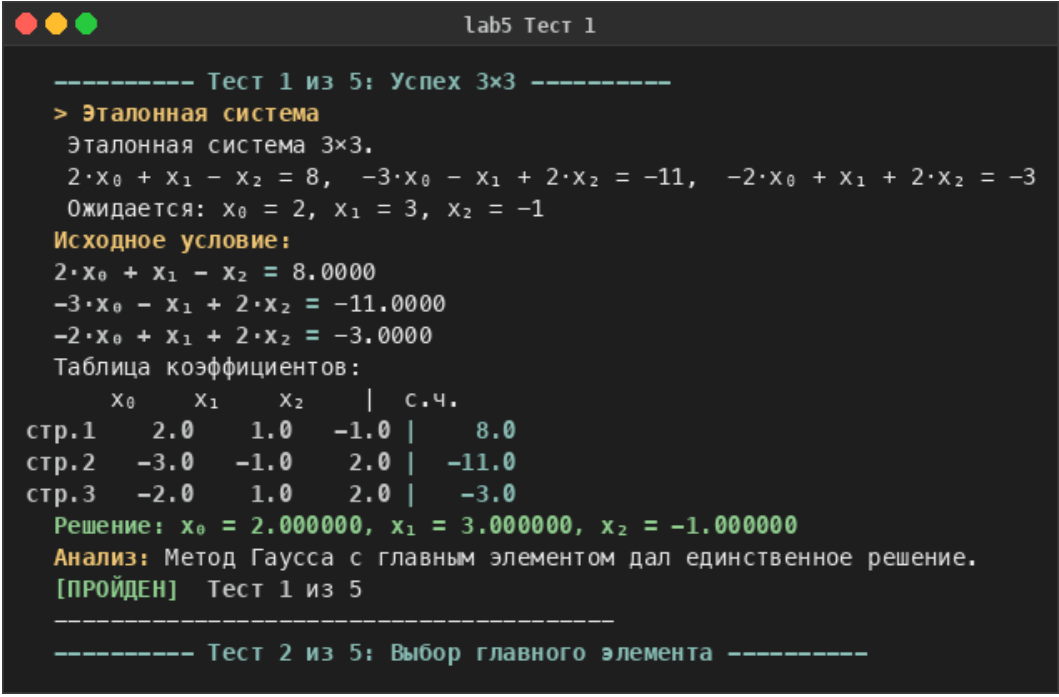
5.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 6 — Автоматические тесты лабораторной работы 5

Тест	Файл описания	Тип	Ожидание
1	01_success_3x3.txt	Успех	$A[3 \times 3], b[3] \rightarrow x = (2, 3, -1)$
2	02_pivot_swap.txt	Успех	A с нулём на диагонали $\rightarrow$ перестановка строк
3	03_bad_sizes.txt	Ошибка	Размеры матрицы и вектора не совпадают
4	04_singular.txt	Ошибка	Вырожденная система (пропорциональные строки)
5	05_inconsistent.txt	Ошибка	Несовместная система (параллельные уравнения)

Тест 1

Эталонная система 3×3 с очевидным решением. Программа выполняет прямой ход с выбором главного элемента и обратный ход; корни совпадают с ожиданием.



```
lab5 Тест 1

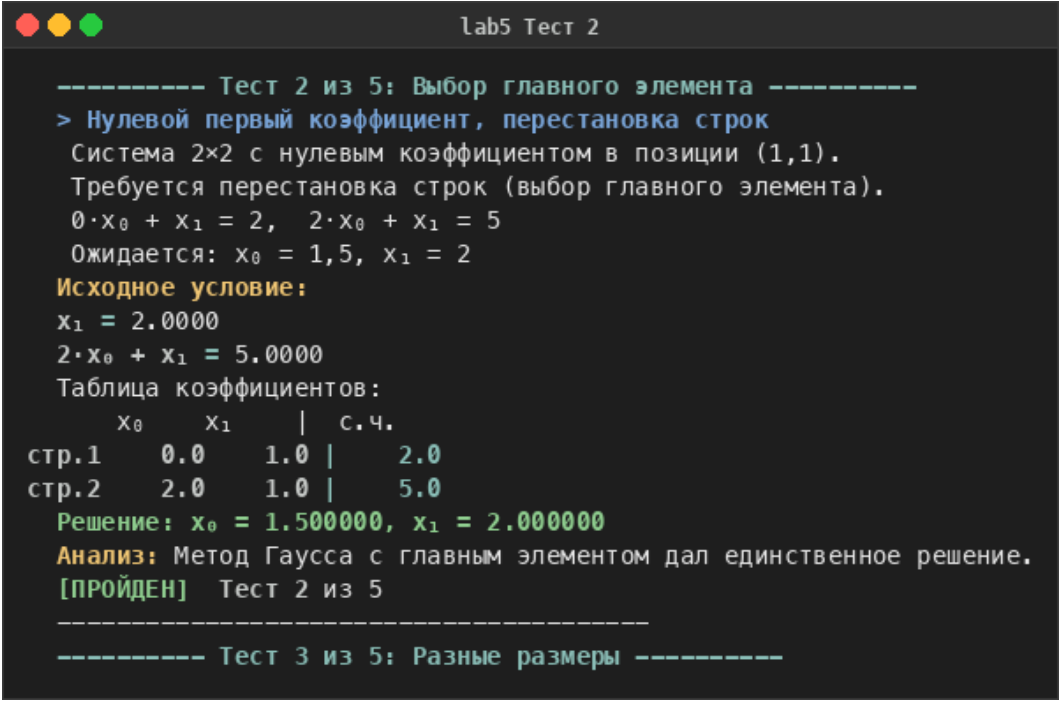
----- Тест 1 из 5: Успех 3x3 -----
> Эталонная система
Эталонная система 3x3.
 $2 \cdot x_0 + x_1 - x_2 = 8$ ,  $-3 \cdot x_0 - x_1 + 2 \cdot x_2 = -11$ ,  $-2 \cdot x_0 + x_1 + 2 \cdot x_2 = -3$ 
Ожидается:  $x_0 = 2$ ,  $x_1 = 3$ ,  $x_2 = -1$ 
Исходное условие:
 $2 \cdot x_0 + x_1 - x_2 = 8.0000$ 
 $-3 \cdot x_0 - x_1 + 2 \cdot x_2 = -11.0000$ 
 $-2 \cdot x_0 + x_1 + 2 \cdot x_2 = -3.0000$ 
Таблица коэффициентов:
      x0      x1      x2      |      с.ч.
стр.1      2.0      1.0     -1.0 |      8.0
стр.2     -3.0     -1.0      2.0 |     -11.0
стр.3     -2.0      1.0      2.0 |      -3.0
Решение:  $x_0 = 2.000000$ ,  $x_1 = 3.000000$ ,  $x_2 = -1.000000$ 
Анализ: Метод Гаусса с главным элементом дал единственное решение.
[ПРОЙДЕН] Тест 1 из 5

----- Тест 2 из 5: Выбор главного элемента -----
```

Рисунок 30 — Вывод автотеста 1 — успешное решение системы 3×3

Тест 2

Первый коэффициент $a[0][0]$ равен нулю, поэтому алгоритм переставляет строки и выбирает вторую строку как опорную. Проверяется корректность pivoting.



```
lab5 Тест 2

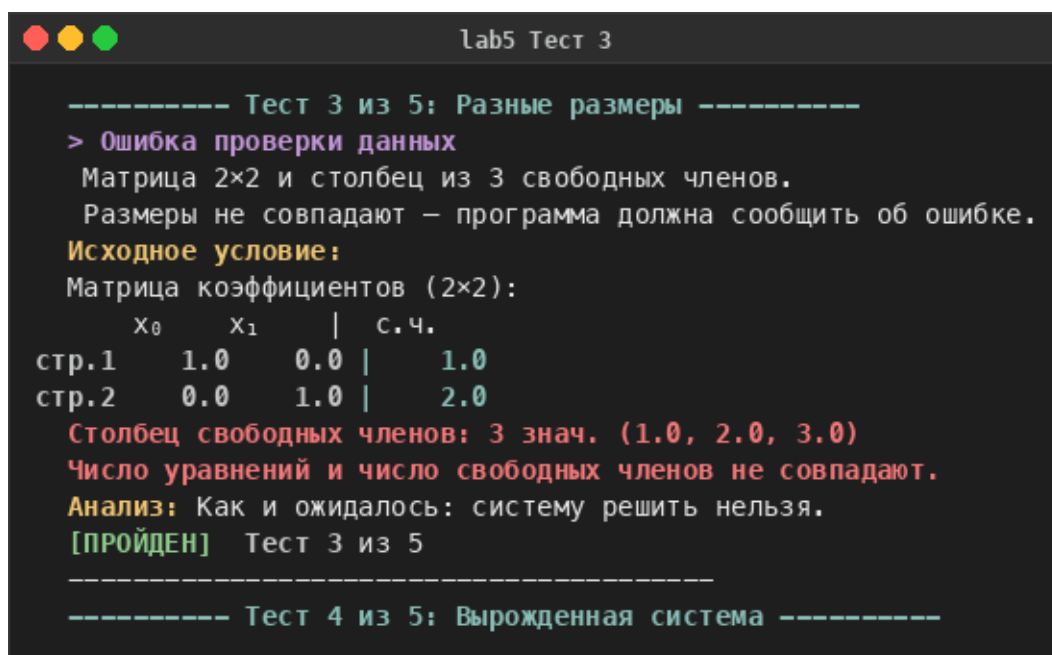
----- Тест 2 из 5: Выбор главного элемента -----
> Нулевой первый коэффициент, перестановка строк
Система 2x2 с нулевым коэффициентом в позиции (1,1).
Требуется перестановка строк (выбор главного элемента).
 $0 \cdot x_0 + x_1 = 2$ ,  $2 \cdot x_0 + x_1 = 5$ 
Ожидается:  $x_0 = 1,5$ ,  $x_1 = 2$ 
Исходное условие:
 $x_1 = 2.0000$ 
 $2 \cdot x_0 + x_1 = 5.0000$ 
Таблица коэффициентов:
       $x_0$     $x_1$    |   С.Ч.
стр.1    0.0    1.0 |    2.0
стр.2    2.0    1.0 |    5.0
Решение:  $x_0 = 1.500000$ ,  $x_1 = 2.000000$ 
Анализ: Метод Гаусса с главным элементом дал единственное решение.
[ПРОЙДЕН] Тест 2 из 5

----- Тест 3 из 5: Разные размеры -----
```

Рисунок 31 — Вывод автотеста 2 — перестановка строк (pivoting)

Тест 3

Матрица 2×2 и вектор из 3 элементов. Программа обнаруживает несовпадение размеров и сообщает об ошибке до попытки решения.



```
lab5 Тест 3

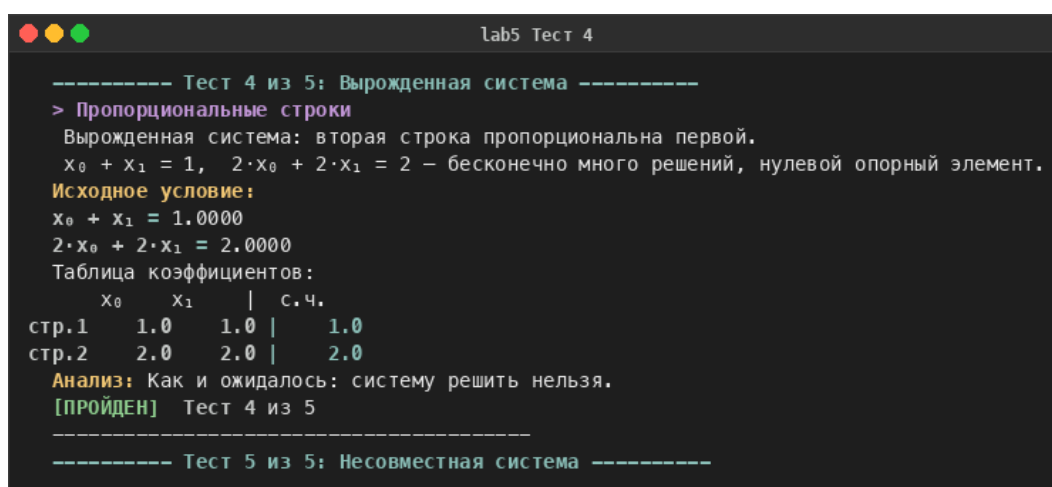
----- Тест 3 из 5: Разные размеры -----
> Ошибка проверки данных
Матрица 2x2 и столбец из 3 свободных членов.
Размеры не совпадают – программа должна сообщить об ошибке.
Исходное условие:
Матрица коэффициентов (2x2):
      x0    x1    |  с.ч.
стр.1   1.0    0.0 |   1.0
стр.2   0.0    1.0 |   2.0
Столбец свободных членов: 3 знач. (1.0, 2.0, 3.0)
Число уравнений и число свободных членов не совпадают.
Анализ: Как и ожидалось: систему решить нельзя.
[ПРОЙДЕН] Тест 3 из 5

----- Тест 4 из 5: Вырожденная система -----
```

Рисунок 32 — Вывод автотеста 3 — несовпадение размеров

Тест 4

Вторая строка матрицы пропорциональна первой. При прямом ходе главный элемент оказывается нулевым, и программа сообщает, что систему решить нельзя.



```
lab5 Тест 4

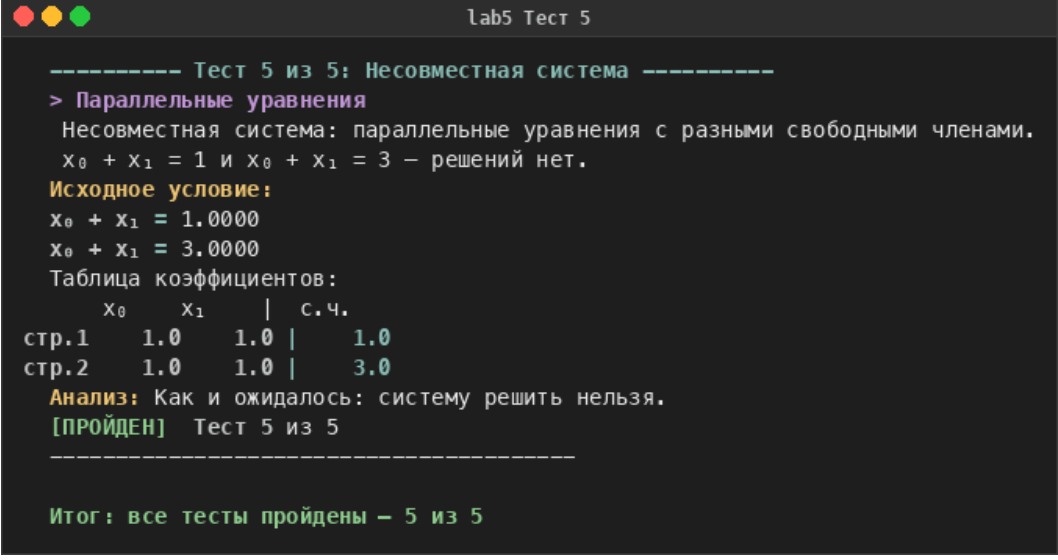
----- Тест 4 из 5: Вырожденная система -----
> Пропорциональные строки
Вырожденная система: вторая строка пропорциональна первой.
x0 + x1 = 1, 2·x0 + 2·x1 = 2 – бесконечно много решений, нулевой опорный элемент.
Исходное условие:
x0 + x1 = 1.0000
2·x0 + 2·x1 = 2.0000
Таблица коэффициентов:
      x0    x1    |  с.ч.
стр.1   1.0    1.0 |   1.0
стр.2   2.0    2.0 |   2.0
Анализ: Как и ожидалось: систему решить нельзя.
[ПРОЙДЕН] Тест 4 из 5

----- Тест 5 из 5: Несовместная система -----
```

Рисунок 33 — Вывод автотеста 4 — вырожденная система

Тест 5

Уравнения параллельны, но правые части различны. Как и в тесте 4, обнаруживается нулевой главный элемент, решение невозможно.



```
lab5 Тест 5

----- Тест 5 из 5: Несовместная система -----
> Параллельные уравнения
Несовместная система: параллельные уравнения с разными свободными членами.
 $x_0 + x_1 = 1$  и  $x_0 + x_1 = 3$  — решений нет.
Исходное условие:
 $x_0 + x_1 = 1.0000$ 
 $x_0 + x_1 = 3.0000$ 
Таблица коэффициентов:
       $x_0$     $x_1$    |   С.Ч.
стр.1    1.0    1.0 |    1.0
стр.2    1.0    1.0 |    3.0
Анализ: Как и ожидалось: систему решить нельзя.
[ПРОЙДЕН] Тест 5 из 5

-----

Итог: все тесты пройдены — 5 из 5
```

Рисунок 34 — Вывод автотеста 5 — несовместная система

6. МЕТОД ТРАПЕЦИЙ И МЕТОД НЬЮТОНА

6.1. ПОСТАНОВКА ЗАДАНИЯ

Вариант 5, повышенная сложность (задание 3.5.2). Требуется вычислить определённый интеграл $I(c) = \int_{[1;2]} \sqrt{(x^2 + c^2)} / x \, dx$ для набора значений параметра c составной формулой трапеций с автоматическим выбором числа разбиений. Для вычисления квадратного корня в подынтегральной функции запрещено использовать `std::sqrt`: значение $\sqrt{t}$ находится как корень уравнения $y^2 - t = 0$ методом Ньютона. Численный интеграл сверяется с аналитическим значением.

Ограничения и требования к программе:

- составную формулу трапеций реализовать для произвольной f , передаваемой указателем на функцию;
- $\sqrt{t}$ вычислять только методом Ньютона по формуле $y_{(k+1)} = y_k - (y_k^2 - t)/(2 \cdot y_k)$;
- критерий остановки метода Ньютона: $|y_{(k+1)} - y_k| < \text{eps_sqrt}$;
- отделение корня уравнения $y^2 - t = 0$ проверять явно (`isRootBracketed`);
- адаптивно удваивать число разбиений $n \rightarrow 2n \rightarrow 4n \rightarrow \dots$ до $|I_{\text{prev}} - I_{\text{curr}}| < \text{eps_integral}$;
- сравнивать численный результат с аналитическим значением интеграла;
- покрыть решение автотестами (трапеции, Ньютон, адаптив, вариант 5).

6.2. АЛГОРИТМ И ПРОГРАММА

6.2.1. Метод Ньютона (*newtonSolve*)

Универсальная реализация метода Ньютона для решения уравнения $f(y) = 0$. Функция принимает указатели на f и её производную df , начальное приближение y_0 и точность eps . Останавливается при $|y_{(k+1)} - y_k| < \text{eps}$

либо при достижении лимита итераций. Дополнительно защищена от деления на ноль при малой производной.

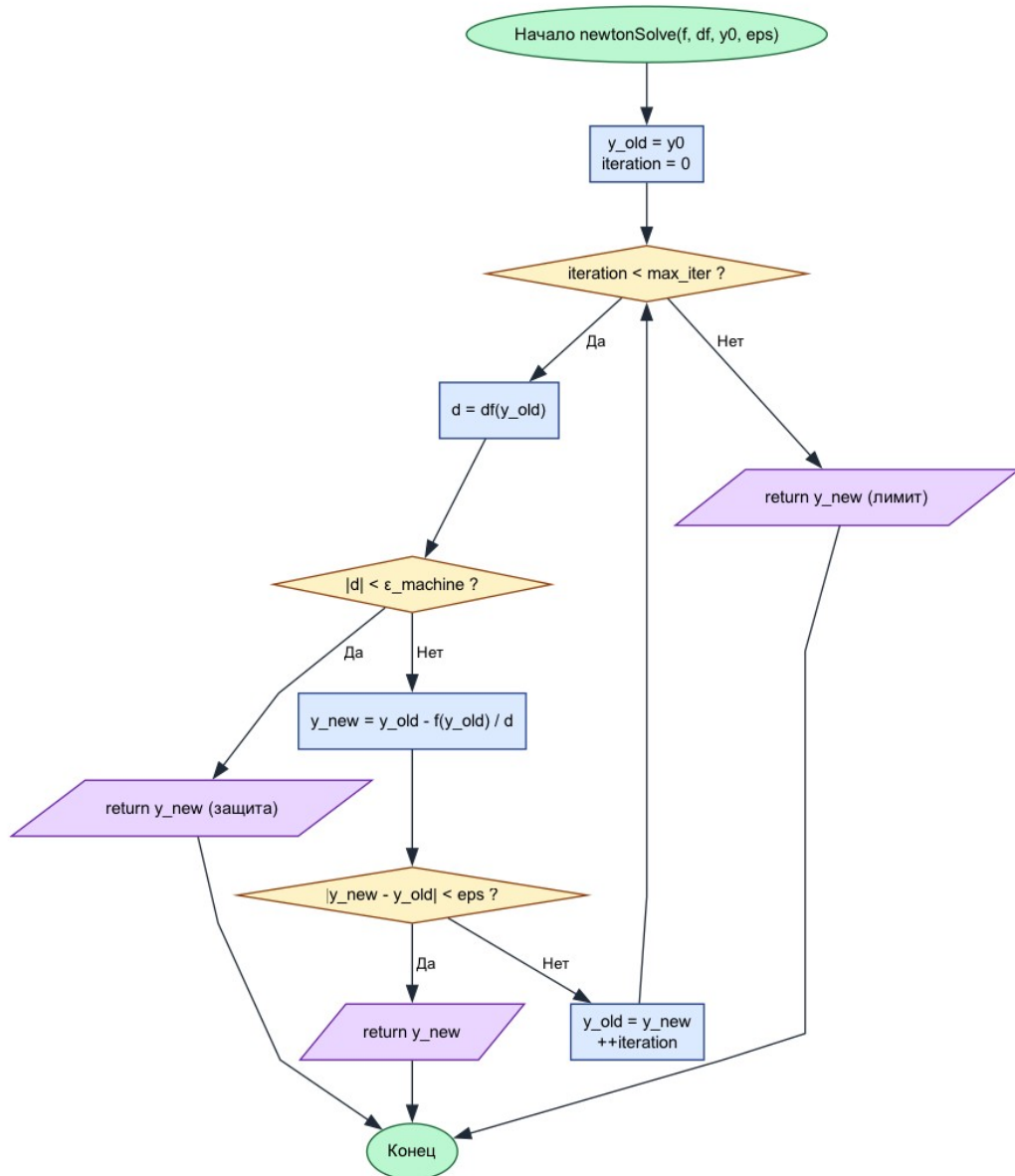


Рисунок 35 — Блок-схема универсального метода Ньютона

```

double newtonSolve(double (*f)(double), double (*df)(double), double y0,
double eps)
{
    // Универсальный шаг метода Ньютона:
    // y_{k+1} = y_k - f(y_k)/f'(y_k)
    // Здесь f и df передаются именно через указатели на функции.
    double y_old = y0;
    double y_new = y0;

    for (int iteration = 0; iteration < max_newton_iterations; ++iteration)
    {

```

```

        const double derivative = df(y_old);
        if (std::abs(derivative) <
std::numeric_limits<double>::epsilon())
        {
            // Если производная почти ноль, делить нельзя — выходим
безопасно.
            break;
        }

        y_new = y_old - f(y_old) / derivative;
        if (std::abs(y_new - y_old) < eps)
        {
            // Критерий остановки из задания:  $|y_{k+1} - y_k| < \text{eps}$ .
            return y_new;
        }
        y_old = y_new;
    }

    return y_new;
}

```

6.2.2. Функции *f_sqrt* и *df_sqrt*

Уравнение $y^2 - t = 0$ представлено в виде функции $f_sqrt(y) = y^2 - t$ с производной $df_sqrt(y) = 2 \cdot y$. Значение t передаётся через глобальную переменную `t_value` — так сохраняется требование передавать функции только по указателю без замыкания параметров.

```

double f_sqrt(double y)
{
    // Левая часть уравнения  $f(y)=0$  для задачи  $\text{sqrt}(t)$ :
    //  $y^2 - t = 0 \Rightarrow y = \text{sqrt}(t)$ .
    return y * y - t_value;
}

double df_sqrt(double y)
{
    // Производная  $f(y)=y^2-t$ :  $f'(y)=2y$ .
    return 2.0 * y;
}

```

6.2.3. Функция *sqrtByNewton*

Оболочка над `newtonSolve` для вычисления $\sqrt{t}$. Проверяет корректность аргумента ($t \geq 0$), отделение корня на $[0; t]$, а затем запускает метод Ньютона с начальным приближением $y_0 = t$. Если $t = 0$ — возвращает 0 сразу.

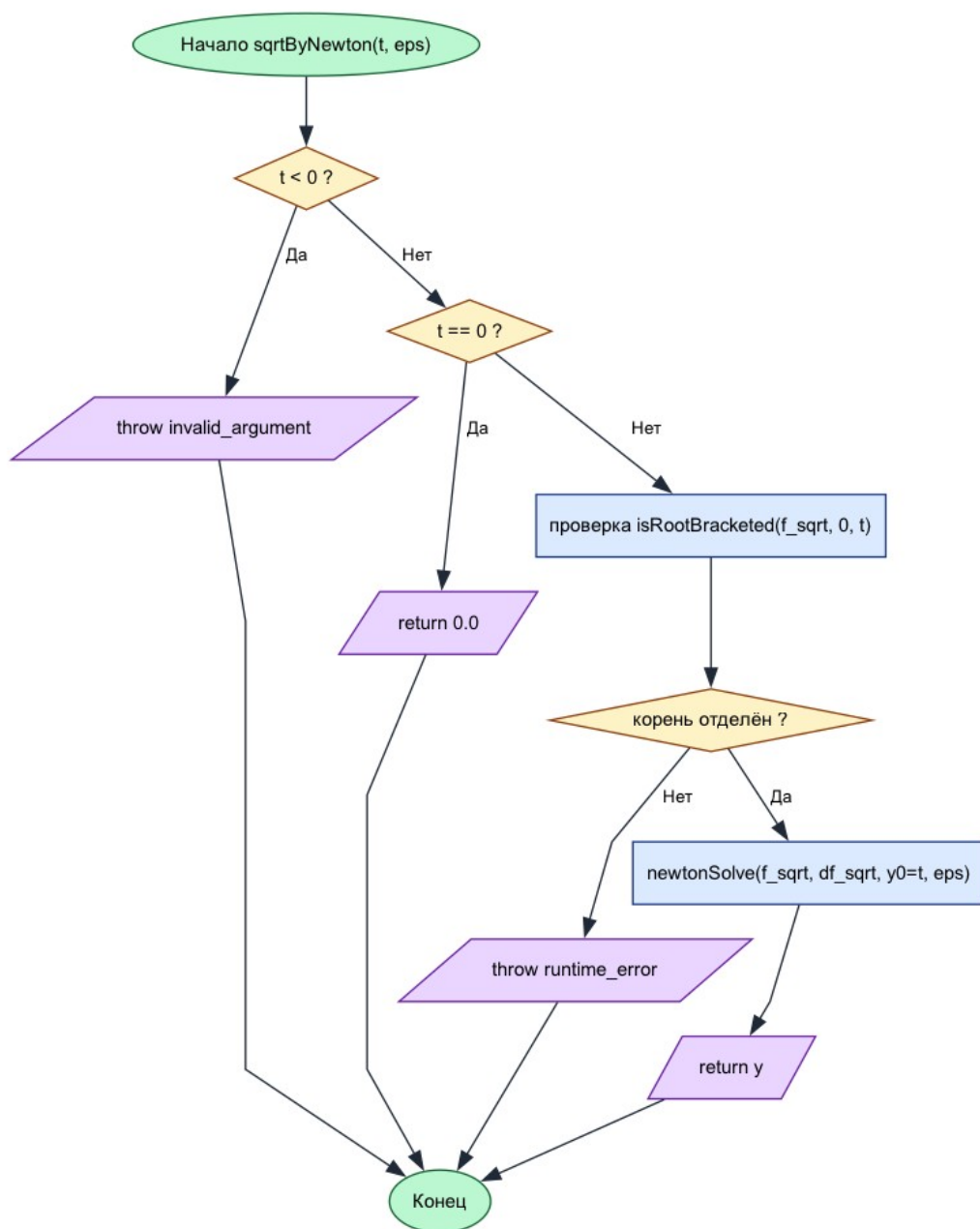


Рисунок 36 — Блок-схема функции *sqrtByNewton*

```

double sqrtByNewton(double t, double eps)
{
    // Эта функция решает  $y^2 - t = 0$  методом Ньютона и возвращает
    sqrt(t).
    t_value = t;
    if (t < 0.0)
    {
        throw std::invalid_argument("sqrtByNewton: параметр t должен
        быть неотрицательным.");
    }
    if (t == 0.0)
    {
        return 0.0;
    }
}

```

```

    }
    if (!isRootBracketed(f_sqrt, 0.0, t))
    {
        // Для t>0 корень sqrt(t) лежит в [0,t]. Проверяем это явно.
        throw std::runtime_error("sqrtByNewton: корень не отделен на
отрезке [0, t].");
    }
    return newtonSolve(f_sqrt, df_sqrt, t, eps);
}

```

6.2.4. Составная формула трапеций (compositeTrapezoid)

Реализация составной формулы трапеций для произвольной подынтегральной функции: $I \approx h \cdot (f(a)/2 + f(a+h) + \dots + f(b-h) + f(b)/2)$, где $h = (b - a) / n$. Функция принимает f через указатель, что позволяет использовать её как для тестовой $f(x) = x$, так и для подынтегральной функции варианта 5.

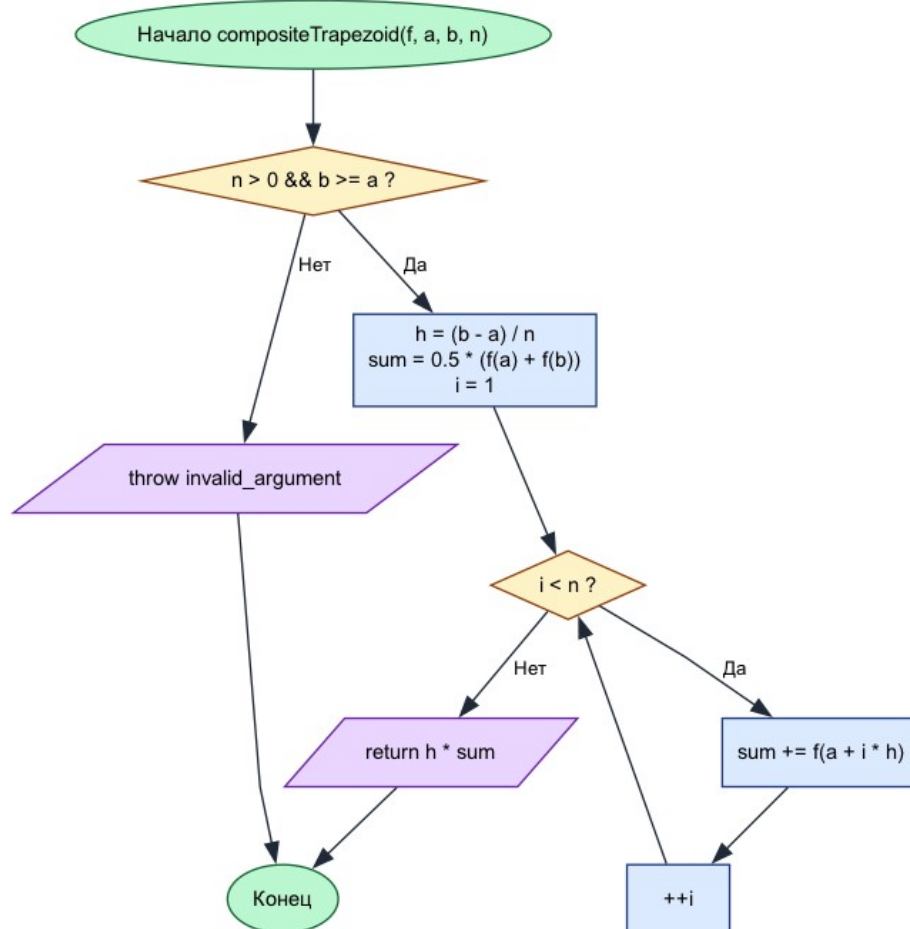


Рисунок 37 — Блок-схема составной формулы трапеций

```

double compositeTrapezoid(double (*f)(double), double a, double b, int
n)
{
    // Составная формула трапеций для произвольной f(x), переданной
указателем:
    //  $I \approx h * ( f(a)/2 + f(a+h) + \dots + f(b)/2 )$ ,  $h=(b-a)/n$ .
    if (n <= 0)
    {
        throw std::invalid_argument("compositeTrapezoid: n должно быть
положительным.");
    }
    if (b < a)
    {
        throw std::invalid_argument("compositeTrapezoid: правая граница
b должна быть >= a.");
    }

    const double h = (b - a) / static_cast<double>(n);
    double sum = 0.5 * (f(a) + f(b));
    for (int i = 1; i < n; ++i)
    {
        sum += f(a + i * h);
    }
    return h * sum;
}

```

6.2.5. Адаптивный метод трапеций (*adaptiveTrapezoid*)

Повышенная сложность задания. Вычисляет интеграл при n , $2n$, $4n$, ... разбиениях и останавливается, когда разность соседних приближений становится меньше `eps_integral`. Возвращает результат вместе со служебной информацией: начальное и итоговое число разбиений, последняя разность, флаг сходимости.

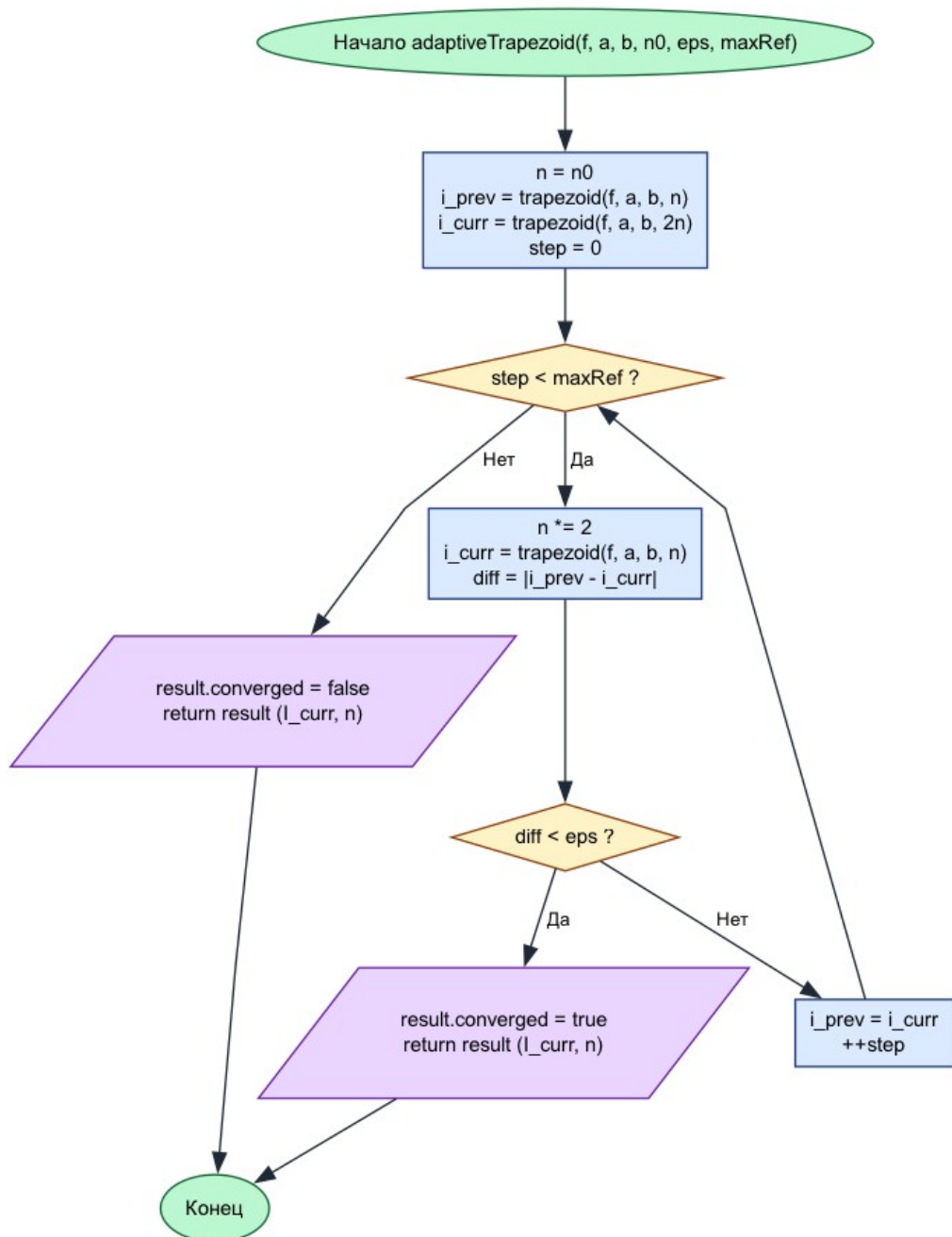


Рисунок 38 — Блок-схема адаптивного метода трапеций

```

AdaptiveResult adaptiveTrapezoid(
    double (*f)(double),
    double a,
    double b,
    int n0,
    double eps_integral,
    int max_refinements)
{
    // "Повышенная сложность" (раздел 3.5.2):
    // считаем интеграл не при одном n, а при n, 2n, 4n, ...
    // останавливаемся, когда |I_prev - I_curr| < eps_integral.
  
```

```

        if (n0 <= 0)
        {
            throw std::invalid_argument("adaptiveTrapezoid: n0 должно быть
положительным.");
        }
        if (eps_integral <= 0.0)
        {
            throw std::invalid_argument("adaptiveTrapezoid: eps_integral
должно быть положительным.");
        }
        if (max_refinements <= 0)
        {
            throw std::invalid_argument("adaptiveTrapezoid: max_refinements
должно быть положительным.");
        }

        int n = n0;
        double i_prev = compositeTrapezoid(f, a, b, n);
        double i_curr = compositeTrapezoid(f, a, b, n * 2);
        double last_diff = std::abs(i_prev - i_curr);

        AdaptiveResult result;
        result.n_start = n0;

        for (int step = 0; step < max_refinements; ++step)
        {
            n *= 2;
            i_curr = compositeTrapezoid(f, a, b, n);
            last_diff = std::abs(i_prev - i_curr);

            if (last_diff < eps_integral)
            {
                result.value = i_curr;
                result.n_final = n;
                result.last_step_diff = last_diff;
                result.converged = true;
                return result;
            }

            i_prev = i_curr;
        }

        result.value = i_curr;
        result.n_final = n;
        result.last_step_diff = last_diff;
        result.converged = false;
        return result;
    }

```

6.2.6. Подынтегральная функция варианта 5

Функция `integrand` возвращает значение $\sqrt{(x^2 + c^2)} / x$, причём корень вычисляется через `sqrtByNewton`, а параметр `c` берётся из глобальной переменной `c_value`. Для аналитической проверки используется первообразная $F(x) = \sqrt{(x^2 + c^2)} + (c/2) \cdot \ln((\sqrt{(x^2 + c^2)} - c)/(\sqrt{(x^2 + c^2)} + c))$.

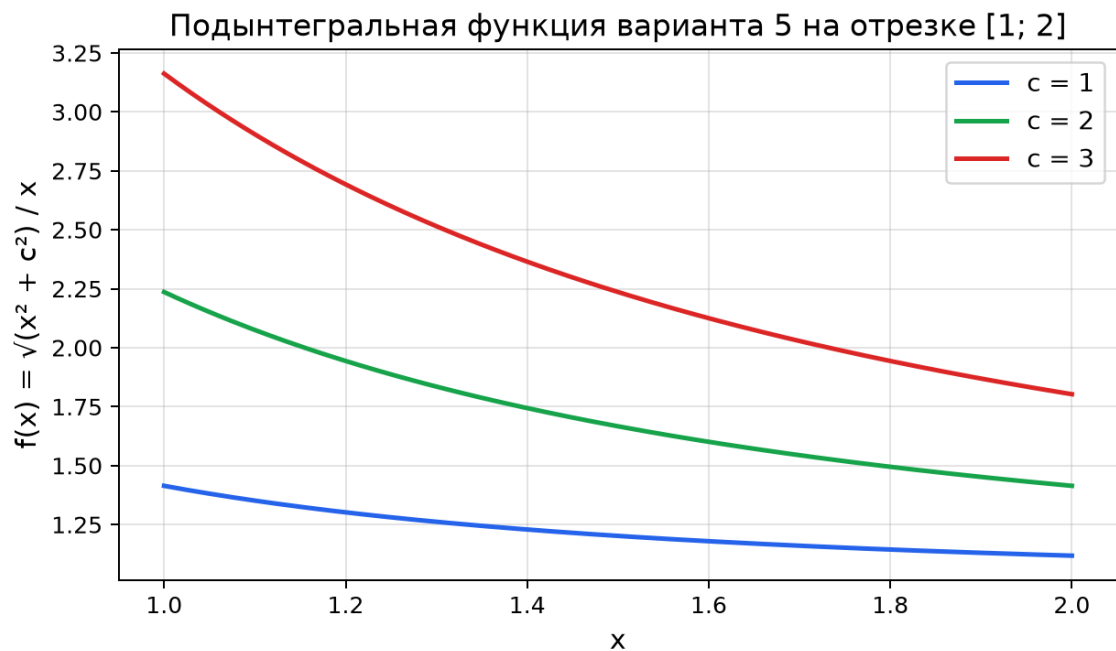


Рисунок 39 — Подынтегральная функция варианта 5 для нескольких значений c

```
double integrand(double x)
{
    // Подынтегральная функция варианта 5:
    // f(x) = sqrt(x^2 + c^2) / x.
    // Корень считаем НЕ std::sqrt, а через sqrtByNewton (требование
    задания).
    const double t = x * x + c_value * c_value;
    return sqrtByNewton(t, sqrt_eps) / x;
}

double primitiveF(double x, double c)
{
    // Первообразная для sqrt(x^2+c^2)/x, нужна для аналитической
    проверки.
    const double root = std::sqrt(x * x + c * c);
    const double numerator = root - c;
    const double denominator = root + c;
    if (numerator <= 0.0 || denominator <= 0.0)
    {
```

```

        throw std::runtime_error("primitiveF: некорректный аргумент
логарифма.");
    }
    return root + (c / 2.0) * std::log(numerator / denominator);
}

double analyticIntegral(double c)
{
    // Аналитическое значение интеграла на [1,2].
    return primitiveF(2.0, c) - primitiveF(1.0, c);
}

```

6.2.7. Сценарий *runVariant5*

Основной сценарий: для каждого c из диапазона 1.0..3.0 с шагом 0.2 запускается `adaptiveTrapezoid`, вычисляется аналитическое значение, ошибка $|I - I_{an}|$ и последняя разность $|dI|$. Результаты выводятся таблицей, что позволяет визуально оценить сходимость метода.

```

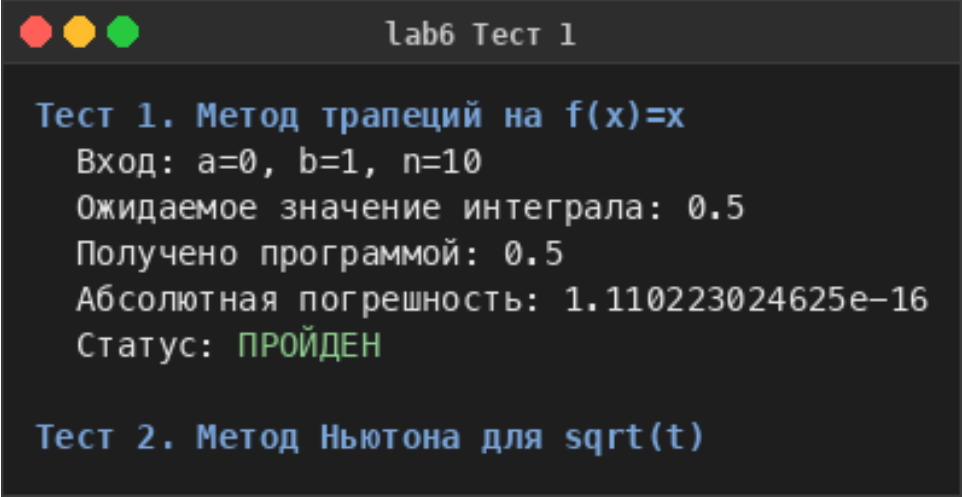
for (double c = 1.0; c <= 3.0 + 1e-9; c += 0.2)
{
    // Для каждого c считаем численное значение, аналитику и ошибки.
    c_value = c;
    const AdaptiveResult adaptive =
        adaptiveTrapezoid(integrand, 1.0, 2.0, cfg.n0,
cfg.eps_integral, cfg.max_refinements);
    const double i_analytic = analyticIntegral(c);
    const double diff_num_an = std::abs(adaptive.value -
i_analytic);

    std::cout << "|"
        << std::setw(6) << std::setprecision(1) << c
        << "|"
        << std::setw(6) << adaptive.n_start
        << "|"
        << std::setw(8) << adaptive.n_final
        << "|"
        << std::setw(12) << std::setprecision(8) <<
adaptive.value
        << "|"
        << std::setw(12) << i_analytic
        << "|"
        << diffColor(diff_num_an) << std::setw(12) <<
diff_num_an << Color::reset
        << "|"
        << diffColor(adaptive.last_step_diff) << std::setw(12)
<< adaptive.last_step_diff
        << Color::reset

```


Тест 1

Проверка формулы трапеций на линейной функции: точный интеграл $\int_0^1 x \, dx = 0.5$. Погрешность в пределах машинного эпсилона — формула реализована корректно.



```
lab6 Тест 1

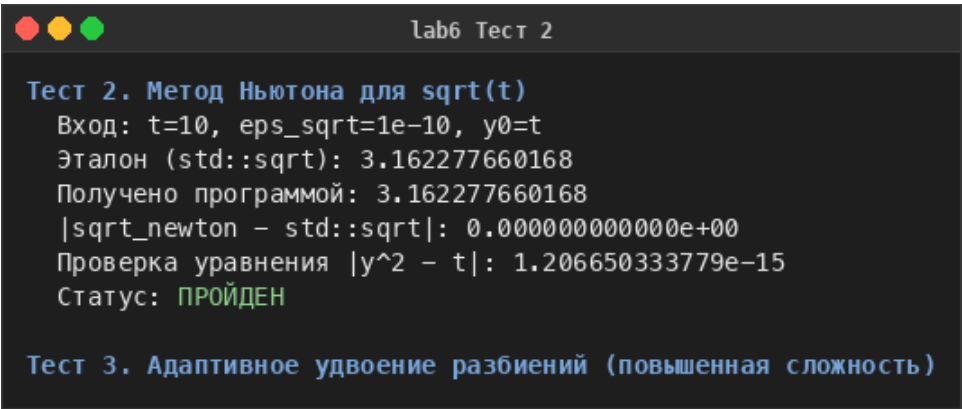
Тест 1. Метод трапеций на f(x)=x
Вход: a=0, b=1, n=10
Ожидаемое значение интеграла: 0.5
Получено программой: 0.5
Абсолютная погрешность: 1.110223024625e-16
Статус: ПРОЙДЕН

Тест 2. Метод Ньютона для sqrt(t)
```

Рисунок 40 — Вывод автотеста 1 — трапеции на $f(x) = x$

Тест 2

Метод Ньютона решает уравнение $y^2 - 10 = 0$ и находит $\sqrt{10}$. Результат совпадает со значением `std::sqrt(10)` с точностью выше запрошенной `eps=1e-10`.



```
lab6 Тест 2

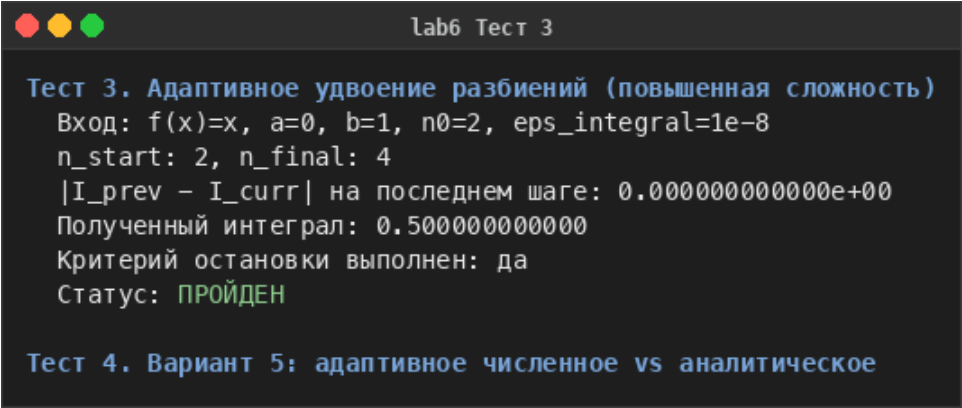
Тест 2. Метод Ньютона для sqrt(t)
Вход: t=10, eps_sqrt=1e-10, y0=t
Эталон (std::sqrt): 3.162277660168
Получено программой: 3.162277660168
|sqrt_newton - std::sqrt|: 0.000000000000e+00
Проверка уравнения |y^2 - t|: 1.206650333779e-15
Статус: ПРОЙДЕН

Тест 3. Адаптивное удвоение разбиений (повышенная сложность)
```

Рисунок 41 — Вывод автотеста 2 — Ньютон для $\sqrt{10}$

Тест 3

Адаптивное удвоение разбиений на линейной функции: критерий $|I_{\text{prev}} - I_{\text{curr}}| < 1e-8$ выполняется уже после первого удвоения, поскольку трапеции точны на линейной f .



```
lab6 Тест 3

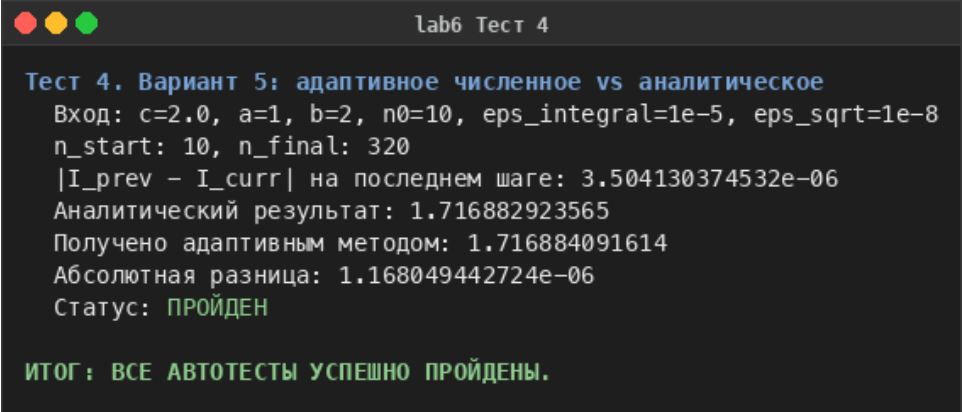
Тест 3. Адаптивное удвоение разбиений (повышенная сложность)
Вход: f(x)=x, a=0, b=1, n0=2, eps_integral=1e-8
n_start: 2, n_final: 4
|I_prev - I_curr| на последнем шаге: 0.000000000000e+00
Полученный интеграл: 0.500000000000
Критерий остановки выполнен: да
Статус: ПРОЙДЕН

Тест 4. Вариант 5: адаптивное численное vs аналитическое
```

Рисунок 42 — Вывод автотеста 3 — адаптив на $f(x) = x$

Тест 4

Сравнение адаптивного численного решения варианта 5 при $c=2$ с аналитическим значением. Разность $|I_{\text{num}} - I_{\text{an}}| \sim 10^{-6}$, что подтверждает корректность связки трапеции + метод Ньютона.



```
lab6 Тест 4

Тест 4. Вариант 5: адаптивное численное vs аналитическое
Вход: c=2.0, a=1, b=2, n0=10, eps_integral=1e-5, eps_sqrt=1e-8
n_start: 10, n_final: 320
|I_prev - I_curr| на последнем шаге: 3.504130374532e-06
Аналитический результат: 1.716882923565
Получено адаптивным методом: 1.716884091614
Абсолютная разница: 1.168049442724e-06
Статус: ПРОЙДЕН

ИТОГ: ВСЕ АВТОТЕСТЫ УСПЕШНО ПРОЙДЕНЫ.
```

Рисунок 43 — Вывод автотеста 4 — вариант 5 vs аналитика

Тест 5

Итоговая таблица результатов основного сценария: для каждого значения c показаны число разбиений n_{fin} (растёт с усложнением подынтегральной функции), численный интеграл I_{num} , аналитический I_{an} , ошибка $|I - I_{an}|$ и критерий остановки $|dI|$. Все погрешности порядка 10^{-7} , что подтверждает работу адаптивного метода.

Lab6 Таблица варианта 5

=== Таблица результатов (адаптивный метод трапеций) ===

	c	n0	n_fin	I_num	I_an	I-Ia	dI
	1.0	8	512	1.22201637	1.22201618	0.00000019	0.00000057
	1.2	8	512	1.30690895	1.30690871	0.00000024	0.00000073
	1.4	8	512	1.40028804	1.40028774	0.00000030	0.00000090
	1.6	8	1024	1.50055767	1.50055758	0.00000009	0.00000026
	1.8	8	1024	1.60643173	1.60643163	0.00000010	0.00000030
	2.0	8	1024	1.71688304	1.71688292	0.00000011	0.00000034
	2.2	8	1024	1.83109453	1.83109440	0.00000013	0.00000038
	2.4	8	1024	1.94841548	1.94841534	0.00000014	0.00000042
	2.6	8	1024	2.06832581	2.06832566	0.00000015	0.00000046
	2.8	8	1024	2.19040772	2.19040755	0.00000016	0.00000049
	3.0	8	1024	2.31432352	2.31432334	0.00000018	0.00000053

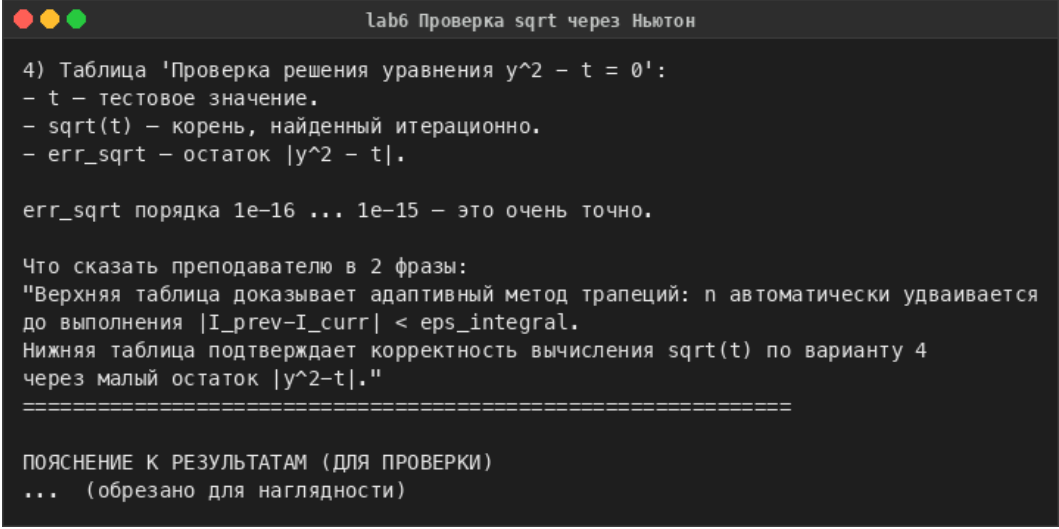
|I-Ia| = |I_num - I_an|, |dI| = |I_prev - I_curr| на последнем шаге, err_sqrt = |y^2 - t|

=== Проверка решения уравнения $y^2 - t = 0$ ===

Рисунок 44 — Таблица результатов основного сценария (адаптивный метод трапеций)

Тест 6

Отдельная таблица проверки решения уравнения $y^2 - t = 0$ для $t = 2, 5, 10$. Остаток $\text{err_sqrt} = |y^2 - t| \sim 10^{-15}$ — метод Ньютона находит корень с предельной точностью.



```
lab6 Проверка sqrt через Ньютон

4) Таблица 'Проверка решения уравнения y^2 - t = 0':
- t - тестовое значение.
- sqrt(t) - корень, найденный итерационно.
- err_sqrt - остаток |y^2 - t|.

err_sqrt порядка 1e-16 ... 1e-15 - это очень точно.

Что сказать преподавателю в 2 фразы:
"Верхняя таблица доказывает адаптивный метод трапеций: n автоматически удваивается
до выполнения |I_prev - I_curr| < eps_integral.
Нижняя таблица подтверждает корректность вычисления sqrt(t) по варианту 4
через малый остаток |y^2 - t|."
=====

ПОЯСНЕНИЕ К РЕЗУЛЬТАТАМ (ДЛЯ ПРОВЕРКИ)
... (обрезано для наглядности)
```

Рисунок 45 — Проверка вычисления $\sqrt{t}$ через метод Ньютона

7. ОДНОНАПРАВЛЕННЫЙ СПИСОК СТУДЕНТОВ

7.1. ПОСТАНОВКА ЗАДАНИЯ

Разработать программу управления однонаправленным связным списком записей о студентах. Каждый элемент списка содержит фамилию, имя и массив из пяти оценок (целые числа от 1 до 10). Программа должна поддерживать создание списка с клавиатуры, сортировку по среднему баллу (по возрастанию или убыванию), а также вставку нового студента с сохранением текущего порядка сортировки.

Ограничения и требования к программе:

- хранить студентов в связном списке (используется `std::list<StudentRecord>`);
- средний балл вычислять как арифметическое среднее пяти оценок;
- проверять оценки на принадлежность диапазону `[1; 10]` (иначе `invalid_argument`);
- сортировка должна быть устойчивой — при равных средних порядок ввода сохраняется;
- вставка выполняется через `std::find_if` с сохранением инварианта отсортированности;
- поддерживать оба порядка (возрастание, убывание) через `std::function`-компаратор;
- предусмотреть автотесты для сортировки и вставки в оба порядка.

7.2. АЛГОРИТМ И ПРОГРАММА

7.2.1. Структура *StudentRecord* и класс *StudentList*

Запись студента реализована как структура с публичными полями фамилии, имени и `std::array<int, 5>` для оценок; метод `averageGrade`

возвращает средний балл. `StudentList` инкапсулирует `std::list<StudentRecord>` и предоставляет методы создания, вывода, сортировки и вставки.



Рисунок 46 — Структура однонаправленного списка `StudentList`

```

{
    std::string surname;
    std::string firstName;
    std::array<int, 5> grades{};

    [[nodiscard]] double averageGrade() const;
};

struct ListNode
{
    StudentRecord data;
    ListNode* next{nullptr};
};

class StudentList
{
public:
    StudentList() = default;
    ~StudentList();

    StudentList(const StudentList&) = delete;
    StudentList& operator=(const StudentList&) = delete;

    void clear();
    [[nodiscard]] bool isEmpty() const;

    void createFromKeyboard();
    void print(std::ostream& os) const;
    void printColored(std::ostream& os) const;

    void append(const StudentRecord& student);
    void sortByAverage(const AverageComparator& compare);
    void insertSorted(const StudentRecord& student, const AverageComparator& compare);
    void insertSortedFromKeyboard(const AverageComparator& compare);

```

7.2.2. Функция `averageGrade` и валидация оценок

`averageGrade` суммирует пять целых оценок и делит на 5.0 — используется как ключ для сортировки и вставки. `validateGrade` выбрасывает `std::invalid_argument` при оценке вне диапазона `[1; 10]`.

```

{
    constexpr int MIN_GRADE = 1;
    constexpr int MAX_GRADE = 10;

    void validateGrade(int grade)
    {
        if (grade < MIN_GRADE || grade > MAX_GRADE)
        {
            throw std::invalid_argument(
                "Ошибка: оценка должна быть в диапазоне [" +
std::to_string(MIN_GRADE) + ", "
                + std::to_string(MAX_GRADE) + "].");
        }
    }
}

double StudentRecord::averageGrade() const
{
    int sum = 0;
    for (int g : grades)
    {
        sum += g;
    }
    return static_cast<double>(sum) / grades.size();
}

```

7.2.3. Компараторы среднего балла

Компаратор задаётся как `std::function<bool(double, double)>`. Функции `ascendingAverage` и `descendingAverage` возвращают готовые лямбды, `readAverageComparatorFromKeyboard` спрашивает у пользователя направление порядка. `comparatorLabel` определяет текущее направление по поведению переданного компаратора.

```

#pragma once

#include <functional>
#include <iostream>

using AverageComparator = std::function<bool(double, double)>;

inline AverageComparator ascendingAverage()
{
    return [](double a, double b) { return a < b; };
}

inline AverageComparator descendingAverage()

```

```

{
    return [](double a, double b) { return a > b; };
}

inline AverageComparator readAverageComparatorFromKeyboard()
{
    int choice = 0;
    std::cout << " 1. По возрастанию\n 2. По убыванию\n Ваш выбор: ";
    std::cin >> choice;
    return choice == 2 ? descendingAverage() : ascendingAverage();
}

inline const char* comparatorLabel(const AverageComparator& comp)
{
    return comp(2.0, 1.0) ? "по убыванию" : "по возрастанию";
}

```

7.2.4. Функция *sortByAverage*

Делегирует сортировку методу `std::list::sort` с лямбдой, вызывающей переданный компаратор от `averageGrade` двух записей. `std::list::sort` реализована как устойчивая сортировка слиянием со сложностью $O(n \log n)$, что подтверждается тестом 3.

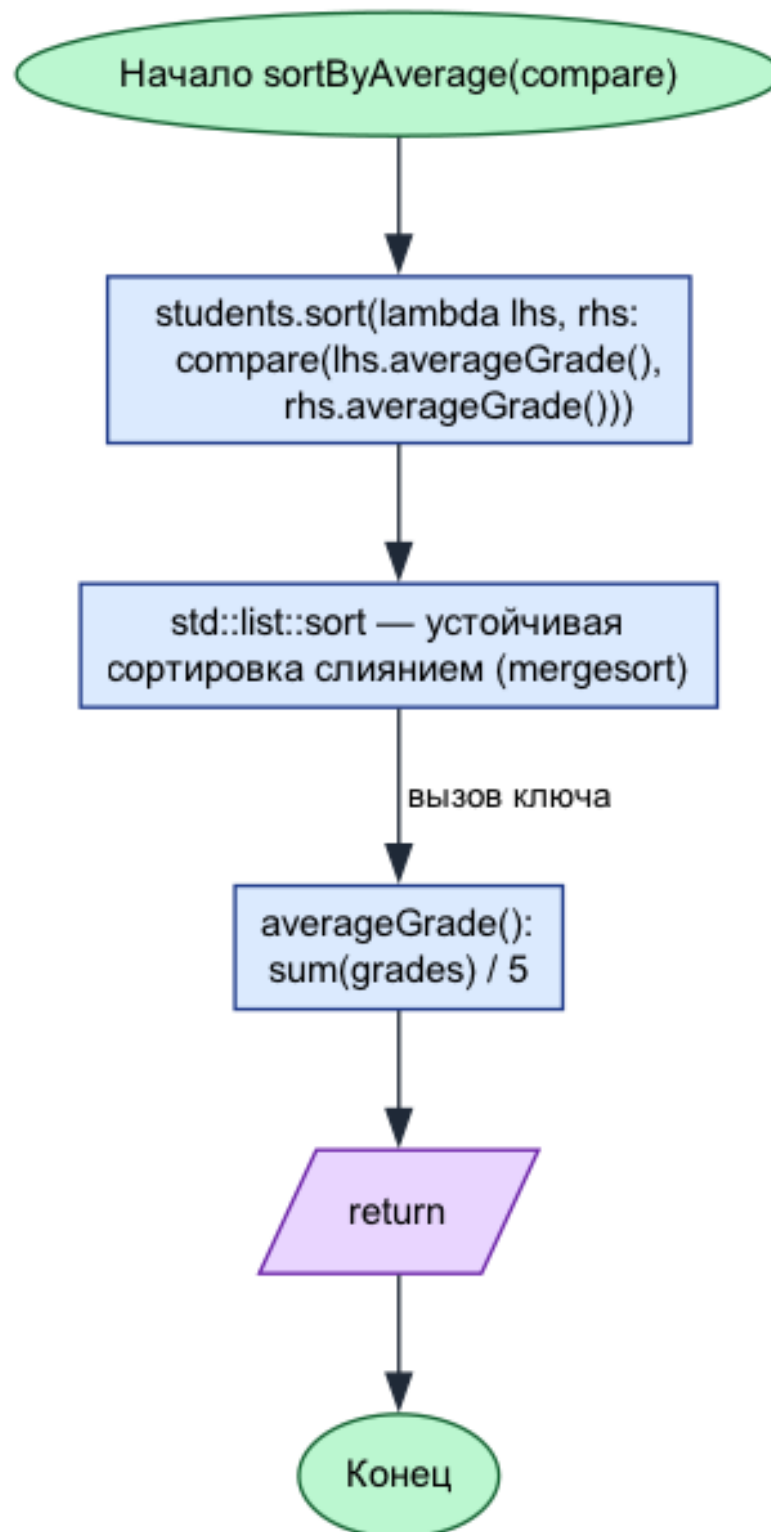


Рисунок 47 — Блок-схема функции `sortByAverage`

```
for (const ListNode* current = head; current != nullptr; current = current->next)
{
    printStudent(current->data, os, index++);
}
```

```
}
```

7.2.5. Функция *insertSorted*

`std::find_if` последовательно перебирает элементы и возвращает первый, для которого `compare(newAvg, currentAvg)` истинно; в эту позицию через `list::insert` вставляется новый студент. Если такого элемента нет, вставка происходит в конец списка.

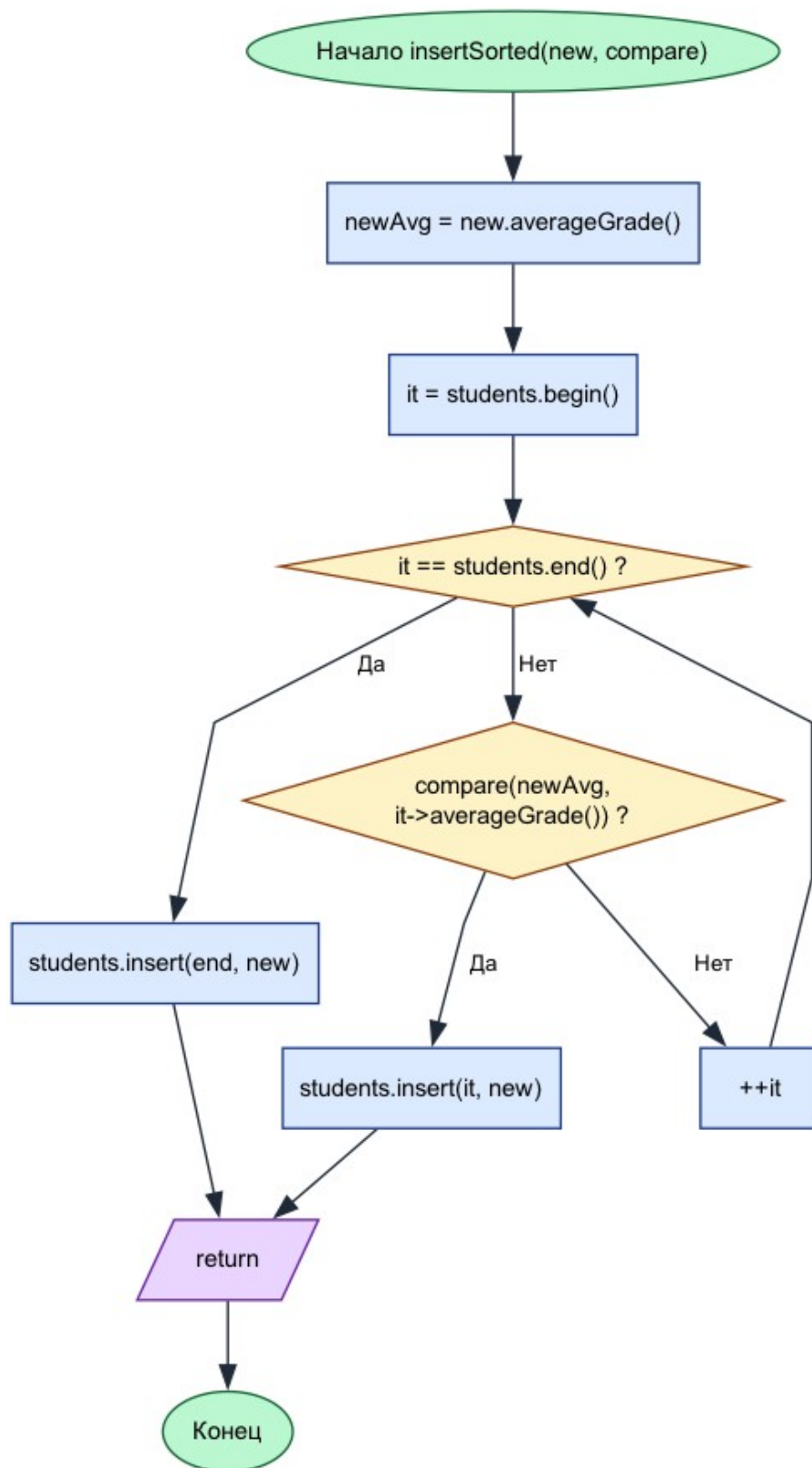


Рисунок 48 — Блок-схема функции *insertSorted*

```

{
    if (isEmpty())
    {

```

```

        os << Color::dim << "  (список пуст)\n" << Color::reset;
        return;
    }

    int index = 1;
    for (const ListNode* current = head; current != nullptr; current =
current->next)
    {
        printStudentColored(current->data, os, index++);
    }
}

```

7.2.6. Функции создания и ввода студента

createFromKeyboard очищает список, запрашивает количество студентов и по одному добавляет их через append. readStudentFromKeyboard читает фамилию, имя и пять оценок; каждая оценка проверяется через validateGrade.

```

{
    return head == nullptr;
}

void StudentList::append(const StudentRecord& student)
{
    ListNode* node = new ListNode{student, nullptr};
    if (!head)
    {
        head = node;
        return;
    }

    ListNode* cur = head;
    while (cur->next)
    {
        cur = cur->next;
    }
    cur->next = node;

    ListNode* prev = sorted;
    while (prev->next != nullptr
        && !compare(current->data.averageGrade(), prev->next-
>data.averageGrade()))
    {
        prev = prev->next;
    }

    current->next = prev->next;
    prev->next = current;
}

```

```

    }

    current = next;
}

head = sorted;
}

void StudentList::insertSorted(const StudentRecord& student, const
AverageComparator& compare)

```

7.3. ТЕСТИРОВАНИЕ ПРОЕКТА

Таблица 8 — Автоматические тесты лабораторной работы 7

Тест	Тип	Что проверяется
1	Сортировка	Четыре различных дробных средних (5.6, 6.8, 7.6, 8.4)
2	Граничный	Список из одного элемента (средний 6.8)
3	Стабильность	Три студента с одинаковым средним 7.2 — сохранение порядка
4	Вставка	Новый минимум (5.4) и максимум (8.8) в отсортированный список
5	Вставка	Вставка в середину (7.2) и с равным ключом (7.6)

Тест 1

Четыре студента с дробными средними баллами располагаются в порядке возрастания. Проверяется базовая сортировка по ключу.

```
lab7 Тест 1 (по возрастанию)

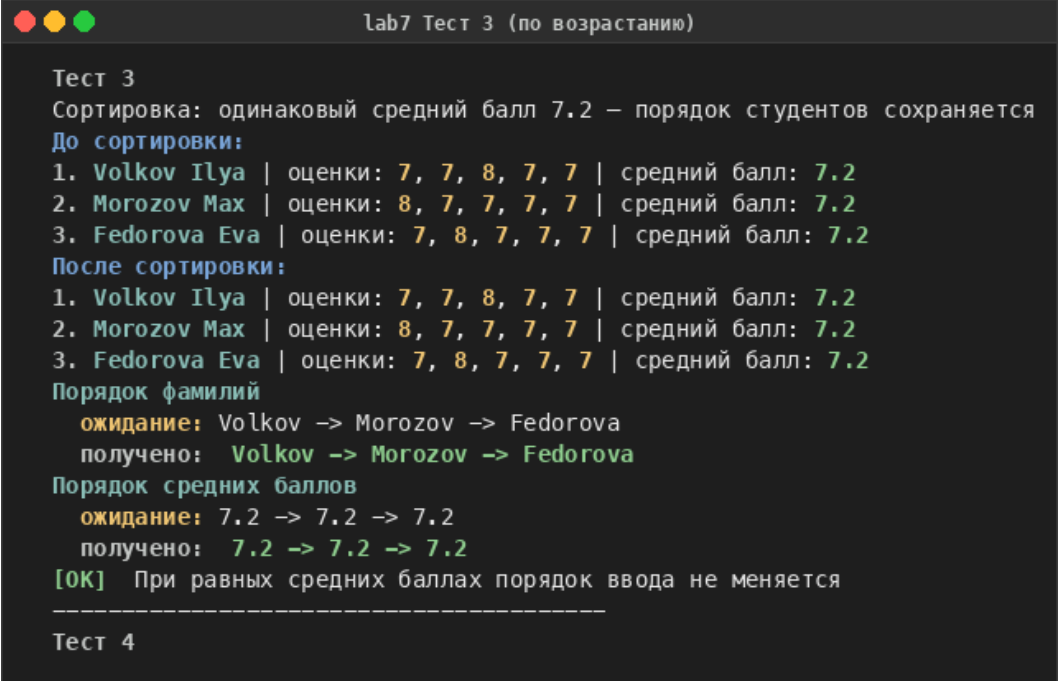
Тест 1
Сортировка: дробные средние (5.6, 6.8, 7.6, 8.4) вперемешку
До сортировки:
1. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
2. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
3. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
4. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
После сортировки:
1. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
2. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
3. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
4. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
Порядок средних баллов
  ожидание: 5.6 -> 6.8 -> 7.6 -> 8.4
  получено:  5.6 -> 6.8 -> 7.6 -> 8.4
[OK] Четыре дробных средних выстроены в выбранном порядке
-----

Тест 2
```

Рисунок 49 — Вывод автотеста 1 (по возрастанию) — сортировка четырёх студентов

Тест 3

Три студента с одинаковым средним баллом 7.2. После сортировки их порядок должен совпадать с порядком ввода — это подтверждает устойчивость `std::list::sort`.



```
lab7 Тест 3 (по возрастанию)

Тест 3
Сортировка: одинаковый средний балл 7.2 – порядок студентов сохраняется
До сортировки:
1. Volkov Ilya | оценки: 7, 7, 8, 7, 7 | средний балл: 7.2
2. Morozov Max | оценки: 8, 7, 7, 7, 7 | средний балл: 7.2
3. Fedorova Eva | оценки: 7, 8, 7, 7, 7 | средний балл: 7.2
После сортировки:
1. Volkov Ilya | оценки: 7, 7, 8, 7, 7 | средний балл: 7.2
2. Morozov Max | оценки: 8, 7, 7, 7, 7 | средний балл: 7.2
3. Fedorova Eva | оценки: 7, 8, 7, 7, 7 | средний балл: 7.2
Порядок фамилий
ожидание: Volkov -> Morozov -> Fedorova
получено: Volkov -> Morozov -> Fedorova
Порядок средних баллов
ожидание: 7.2 -> 7.2 -> 7.2
получено: 7.2 -> 7.2 -> 7.2
[OK] При равных средних баллах порядок ввода не меняется
-----

Тест 4
```

Рисунок 50 — Вывод автотеста 3 (по возрастанию) — устойчивость сортировки

Тест 4

В уже отсортированный список из четырёх студентов вставляются новый минимум (средний 5.4) и новый максимум (средний 8.8). Проверяется корректность insertSorted для граничных позиций.

```
lab7 Тест 4 (по возрастанию)

Тест 4
Вставка: новый минимум (5.4) и новый максимум (8.8) в отсортированный список
Исходный отсортированный список:
1. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
2. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
3. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
4. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
После вставки Smirnov (5.4) и Popov (8.8):
1. Smirnov Stepan | оценки: 5, 5, 6, 5, 6 | средний балл: 5.4
2. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
3. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
4. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
5. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
6. Popov Pavel | оценки: 9, 9, 8, 9, 9 | средний балл: 8.8
Порядок средних баллов
ожидание: 5.4 -> 5.6 -> 6.8 -> 7.6 -> 8.4 -> 8.8
получено: 5.4 -> 5.6 -> 6.8 -> 7.6 -> 8.4 -> 8.8
[OK] Вставка на края списка сохраняет выбранный порядок
-----

Тест 5
```

Рисунок 51 — Вывод автотеста 4 (по возрастанию) — вставка на края списка

Тест 5

Вставка в середину списка (средний 7.2) и вставка студента с уже существующим средним (7.6). Дубликат ключа добавляется корректно, инвариант сохраняется.

```
Lab7 Тест 5 (по возрастанию)

Тест 5
Вставка: в середину (7.2) и студент с тем же средним (7.6)
Исходный отсортированный список:
1. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
2. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
3. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
4. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
После вставки Lebedev (7.2) и Gromov (7.6):
1. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
2. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
3. Lebedev Lev | оценки: 7, 8, 7, 7, 7 | средний балл: 7.2
4. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
5. Gromov Grigory | оценки: 8, 8, 7, 7, 8 | средний балл: 7.6
6. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
Порядок средних баллов
  ожидание: 5.6 -> 6.8 -> 7.2 -> 7.6 -> 7.6 -> 8.4
  получено: 5.6 -> 6.8 -> 7.2 -> 7.6 -> 7.6 -> 8.4
[OK] Вставка в середину и при равном среднем балле
-----

ИТОГ: все тесты пройдены – 5 из 5
```

Рисунок 52 — Вывод автотеста 5 (по возрастанию) — вставка в середину и при равном ключе

Тест 6

Тот же тест 1, но с компаратором убывания. Результат — обратный порядок средних баллов, что подтверждает работу компаратора-стратегии.

```
lab7 Тест 1 (по убыванию)

Тест 1
Сортировка: дробные средние (5.6, 6.8, 7.6, 8.4) вперемешку
До сортировки:
1. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
2. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
3. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
4. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
После сортировки:
1. Orlov Oleg | оценки: 9, 8, 8, 8, 9 | средний балл: 8.4
2. Kuznetsova Anna | оценки: 8, 7, 8, 8, 7 | средний балл: 7.6
3. Belova Daria | оценки: 7, 7, 6, 7, 7 | средний балл: 6.8
4. Nguyen Nam | оценки: 6, 5, 6, 5, 6 | средний балл: 5.6
Порядок средних баллов
ожидание: 8.4 -> 7.6 -> 6.8 -> 5.6
получено: 8.4 -> 7.6 -> 6.8 -> 5.6
[OK] Четыре дробных средних выстроены в выбранном порядке
-----

Тест 2
```

Рисунок 53 — Вывод автотеста 1 (по убыванию) — сортировка в обратном порядке